MODUL 08 · STATE MANAGEMENT

# Manajemen State Terstruktur dengan GetX & Pola MVC

Memahami batasan setState() dan bahaya 'prop drilling' pada aplikasi skala besar. Menguasai GetX dengan pola MVC (Model-View-Controller): GetxController, state reaktif .obs dan Obx, dependency injection Get.put/Get.find/Get.lazyPut, GetBuilder, lifecycle onInit/onClose, Bindings, pola state async (isLoading/error/data), komunikasi antar controller dengan workers, struktur folder models/services/controllers/views, auth gate, serta kesalahan umum dan cara memperbaikinya.

**Durasi** Pertemuan 8 (150 Menit) + Belajar Mandiri

**Prasyarat** Modul 07: Navigasi & Routing

## Capaian Pembelajaran

- Menganalisis perbedaan Ephemeral State (State Lokal) vs App State (State Global).
- Menjelaskan pembagian tanggung jawab Model, View, dan Controller pada pola MVC.
- Membuat GetxController dengan state reaktif .obs dan menampilkannya lewat Obx.
- Menyediakan dan mengambil controller dengan Get.put, Get.find, Get.lazyPut, dan Bindings.
- Mengoptimalkan rebuild dengan Obx kecil, GetX, dan GetBuilder + update().
- Memanfaatkan lifecycle onInit/onClose dan menerapkan pola state async isLoading/error/data.
- Menghubungkan beberapa controller dan bereaksi terhadap perubahan state dengan worker ever dan debounce.
- Menyusun struktur folder models/services/controllers/views dan auth gate berbasis GetX.
- Mengenali dan memperbaiki kesalahan umum: Obx tanpa observable, lupa .value, Get.find sebelum Get.put, mengubah isi objek di RxList tanpa refresh.

## Daftar Isi Modul


1. Masalah 'Prop Drilling' & Kapan Butuh State Management?
2. Mengenal GetX & Pola MVC
3. GetxController, .obs & Obx: State Reaktif dari Nol
4. Dependency Injection: Get.put, Get.find & Get.lazyPut
5. Obx, GetX & GetBuilder: Mengatur Rebuild
6. Menyusun MVC: Contoh KRS (Model, View, Controller)

7. Lifecycle Controller & Bindings
8. State Async: isLoading, error, data
9. Komunikasi Antar Controller & Workers
10. Struktur Folder MVC & Auth Gate
11. Kesalahan Umum & Cara Memperbaikinya
12. Studi Kasus: Aplikasi Tugas Kuliah Multi-Halaman
13. Tugas Praktikum
14. Kuis Pemahaman & Kunci Jawaban

## 1. Masalah 'Prop Drilling' & Kapan Butuh State Management?

Ketika aplikasi Flutter tumbuh besar dan memiliki puluhan halaman, mengandalkan `setState()` saja akan menimbulkan masalah serius yang disebut **Prop Drilling**.

### Dua jenis state

| Jenis | Contoh | Alat yang cocok |
|---|---|---|
| **Ephemeral state** (state lokal) | Tab aktif, isi TextField, checkbox "ingat saya", animasi | `setState` di `StatefulWidget` |
| **App state** (state bersama) | User yang login, isi keranjang/KRS, daftar todo, tema gelap | State management, di kursus ini **GetX** dengan pola MVC |

Aturan praktis: jika data hanya dipakai satu widget, `setState` sudah cukup. Jika data dipakai oleh beberapa widget yang berjauhan atau beberapa halaman, saatnya memindahkan data itu ke sebuah **controller** yang dikelola GetX.

**Masalah Prop Drilling vs Solusi GetX**

❌ **Tanpa State Management (Prop Drilling)**

Jika Widget Anak di tingkat ke-5 membutuhkan data Mahasiswa yang ada di Root, data tersebut harus dioper satu per satu melewati Widget Tingkat 1 → 2 → 3 → 4 → 5 lewat konstruktor! Widget perantara terpaksa memegang data yang tidak mereka gunakan.

✅ **Dengan GetX (Controller + Dependency Injection)**

Data disimpan di sebuah **controller** yang didaftarkan sekali dengan `Get.put()`. Widget mana pun, di halaman mana pun, bisa mengambilnya dengan `Get.find()` dan otomatis diperbarui lewat `Obx`, tanpa melewati widget perantara!

### Contoh kecil: data dititipkan dari level ke level

```
// Level 1 punya data
class HomePage extends StatelessWidget {
  final int jumlahKeranjang;
  const HomePage({super.key, required this.jumlahKeranjang});

  @override
```

```
  Widget build(BuildContext context) => Header(jumlahKeranjang: jumlahKeranjang);
}

// Level 2 TIDAK memakai data, hanya meneruskan
class Header extends StatelessWidget {
  final int jumlahKeranjang;
  const Header({super.key, required this.jumlahKeranjang});

  @override
  Widget build(BuildContext context) => IkonKeranjang(jumlah: jumlahKeranjang);
}

// Level 3 yang benar-benar butuh data
class IkonKeranjang extends StatelessWidget {
  final int jumlah;
  const IkonKeranjang({super.key, required this.jumlah});

  @override
  Widget build(BuildContext context) => Text('🛒 $jumlah');
}
```

`Header` sama sekali tidak memakai `jumlahKeranjang`, tetapi tetap wajib menerimanya hanya untuk diteruskan ke `IkonKeranjang`.

### Contoh lengkap: katalog dan keranjang tanpa state management

Jalankan kode ini dan hitung berapa constructor yang menerima data "titipan". Kode ini sengaja belum memakai GetX sama sekali.

**lib/main.dart**

```
// lib/main.dart — salin SELURUH kode ini untuk menggantikan isi lib/main.dart
// Contoh TANPA state management: perhatikan berapa banyak data yang "dititipkan"
import 'package:flutter/material.dart';

void main() {
  runApp(const MyApp());
}

class MyApp extends StatelessWidget {
  const MyApp({super.key});

  @override
  Widget build(BuildContext context) {
    return MaterialApp(
      debugShowCheckedModeBanner: false,
      theme: ThemeData(colorSchemeSeed: Colors.indigo),
      home: const KatalogPage(),
    );
  }
}

// Pemilik state: semua data keranjang ada di sini
class KatalogPage extends StatefulWidget {
  const KatalogPage({super.key});

  @override
```

```dart
  State<KatalogPage> createState() => _KatalogPageState();
}

class _KatalogPageState extends State<KatalogPage> {
  final List<String> _keranjang = [];

  void _tambah(String matkul) => setState(() => _keranjang.add(matkul));

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      // Titipan 1: jumlah dioper ke AppBarKatalog
      appBar: AppBarKatalog(jumlah: _keranjang.length, isi: _keranjang),
      // Titipan 2: callback dioper ke DaftarMatkul -> ItemMatkul
      body: DaftarMatkul(onTambah: _tambah),
    );
  }
}

class AppBarKatalog extends StatelessWidget implements PreferredSizeWidget {
  final int jumlah;
  final List<String> isi;

  const AppBarKatalog({super.key, required this.jumlah, required this.isi});

  @override
  Size get preferredSize => const Size.fromHeight(kToolbarHeight);

  @override
  Widget build(BuildContext context) {
    return AppBar(
      title: const Text('Katalog (tanpa GetX)'),
      // Titipan 3: data dioper lagi ke TombolKeranjang
      actions: [TombolKeranjang(jumlah: jumlah, isi: isi)],
    );
  }
}

class TombolKeranjang extends StatelessWidget {
  final int jumlah;
  final List<String> isi;

  const TombolKeranjang({super.key, required this.jumlah, required this.isi});

  @override
  Widget build(BuildContext context) {
    return IconButton(
      tooltip: 'Keranjang',
      icon: Badge(label: Text('$jumlah'), child: const Icon(Icons.shopping_cart)),
      onPressed: () {
        // Titipan 4: data dioper ke halaman lain lewat constructor
        Navigator.push(
          context,
          MaterialPageRoute(builder: (_) => KeranjangPage(isi: isi)),
        );
      },
    );
  }
}
```

```dart
class DaftarMatkul extends StatelessWidget {
  final ValueChanged<String> onTambah; // tidak dipakai di sini, hanya diteruskan

  const DaftarMatkul({super.key, required this.onTambah});

  static const matkul = ['Pemrograman Mobile', 'Basis Data', 'Jaringan Komputer'];

  @override
  Widget build(BuildContext context) {
    return ListView(
      children: [
        for (final m in matkul) ItemMatkul(nama: m, onTambah: onTambah),
      ],
    );
  }
}

class ItemMatkul extends StatelessWidget {
  final String nama;
  final ValueChanged<String> onTambah;

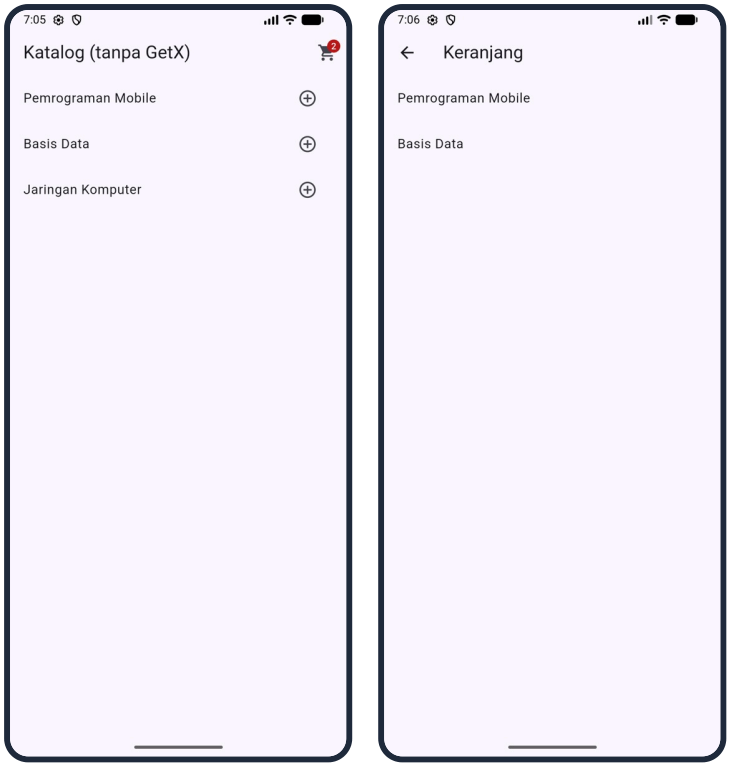
  const ItemMatkul({super.key, required this.nama, required this.onTambah});

  @override
  Widget build(BuildContext context) {
    return ListTile(
      title: Text(nama),
      trailing: IconButton(
        tooltip: 'Tambah $nama',
        icon: const Icon(Icons.add_circle_outline),
        onPressed: () => onTambah(nama), // akhirnya dipakai di level terdalam
      ),
    );
  }
}

class KeranjangPage extends StatelessWidget {
  final List<String> isi;

  const KeranjangPage({super.key, required this.isi});

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(title: const Text('Keranjang')),
      // Masalah tambahan: halaman ini tidak bisa MENGHAPUS item
      // tanpa menitipkan callback lagi dari KatalogPage.
      body: ListView(children: [for (final m in isi) ListTile(title: Text(m))]),
    );
  }
}
```




Katalog: badge menerima jumlah lewat constructor

Keranjang menerima isi lewat constructor

**Analisis masalah:**

- `AppBarKatalog` dan `DaftarMatkul` hanya menjadi "kurir": menerima data/callback lalu meneruskannya.
- Menambah fitur hapus di `KeranjangPage` berarti menambah callback baru yang harus melewati beberapa widget lagi.
- `KeranjangPage` menerima *salinan referensi* list saat dibuka. Jika list berubah, halaman itu tidak tahu harus rebuild karena tidak ada yang memanggil `setState` di sana.
- Setiap `setState` di `KatalogPage` membangun ulang seluruh halaman, termasuk bagian yang tidak berubah.
- Logika (menambah item, nanti mungkin mengecek batas) bercampur dengan kode tampilan di dalam `State`, sehingga sulit diuji tanpa menjalankan UI.

**Bagaimana GetX menyelesaikannya?**

Gambaran singkatnya (detailnya dibahas di section-section berikut):

1. Data keranjang dan method `tambah()` dipindah ke class `KeranjangController`.
2. Controller didaftarkan sekali dengan `Get.put(KeranjangController())`.
3. `TombolKeranjang`, `ItemMatkul`, dan `KeranjangPage` masing-masing mengambil controller itu sendiri dengan `Get.find()`. Widget perantara tidak perlu lagi menerima data titipan.
4. Bagian UI yang menampilkan data dibungkus `Obx`, sehingga hanya bagian itu yang dibangun ulang ketika data berubah, di halaman mana pun ia berada.

### ℹ️ Apa yang akan kita pelajari

Di section berikutnya kita berkenalan dengan GetX dan pola **MVC** (Model-View-Controller). Setelah itu kita membangun solusinya selangkah demi selangkah: `GetxController` dan `.obs` (tempat state), `Obx` (menampilkan state), `Get.put` / `Get.find` (menyediakan dan mengambil controller dari halaman mana pun), lalu cara mengatur rebuild dengan `Obx`, `GetX`, dan `GetBuilder`.

### ⚠️ Kesalahan umum

- Langsung memakai GetX untuk *semua* state. Isi TextField atau tab aktif tetap lebih sederhana dengan `setState`.
- Menyimpan state bersama di variabel global biasa. UI tidak akan otomatis rebuild ketika nilainya berubah.

## 2. Mengenal GetX & Pola MVC

**GetX** (package `get`) adalah library Flutter yang menyediakan state management, dependency injection, dan navigasi dalam satu paket. Di kursus ini GetX selalu dipakai bersama pola **MVC** agar kode tetap rapi meskipun aplikasi membesar.

### Tiga pilar GetX

| Pilar | Fungsinya | API yang akan sering dipakai |
|---|---|---|
| **State management** | Menyimpan state di controller dan membangun ulang UI ketika state berubah | `GetxController`, `.obs`, `Obx`, `GetBuilder` + `update()` |
| **Dependency management** | Mendaftarkan objek sekali lalu mengambilnya dari mana saja, tanpa constructor berantai | `Get.put`, `Get.find`, `Get.lazyPut`, `Bindings` |
| **Route management** | Berpindah halaman tanpa perlu `BuildContext` | `Get.to`, `Get.back`, `Get.off`, `Get.toNamed` |

Ketiganya bisa dipakai terpisah. Fokus modul ini adalah dua pilar pertama; navigasi GetX cukup dipakai seperlunya (`Get.to` dan `Get.back`).

### Pola MVC: membagi tanggung jawab

MVC membagi kode menjadi tiga bagian dengan tugas yang jelas:

| Bagian | Tanggung jawab | Boleh mengimpor | Contoh |
|---|---|---|---|
| **Model** | Bentuk data dan aturan kecil yang melekat pada data (mis. `toJson`, getter turunan) | Dart murni saja | `MataKuliah`, `Tugas`, `Mahasiswa` |
| **View** | Menampilkan state dan meneruskan aksi pengguna ke controller. Juga urusan tampilan: SnackBar, dialog, navigasi | `material.dart`, `get.dart`, controller | `KatalogKrsView`, `LoginView` |
| **Controller** | Menyimpan state (reaktif), menjalankan logika bisnis, memanggil service | `get.dart`, `foundation.dart`, model, service | `KrsController`, `AuthController` |

Untuk akses data (HTTP, database, Hive, Supabase) kita menambahkan satu lapisan lagi, yaitu **Service**. Arah ketergantungannya selalu satu arah:

```
View  ⟶  Controller  ⟶  Service  ⟶  Model
(UI)       (state+logika)    (ambil/simpan data)  (bentuk data)

View mengenal Controller, Controller mengenal Service,
tetapi TIDAK sebaliknya: Controller tidak pernah mengimpor View.
```

### Aturan penting: controller tanpa UI

Di kursus ini controller **tidak** mengimpor `package:flutter/material.dart`, **tidak** menyimpan `BuildContext`, dan **tidak** menampilkan SnackBar, dialog, atau berpindah halaman. Controller cukup *mengembalikan hasil* (misalnya `bool` atau pesan error), lalu View yang memutuskan cara menampilkannya.

```dart
// Controller: hanya mengembalikan hasil
bool tambahMatkul(MataKuliah mk) {
  if (totalSks + mk.sks > 24) return false;
  _matkul.add(mk);
  return true;
}

// View: memutuskan bagaimana memberi tahu pengguna
onPressed: () {
  if (!controller.tambahMatkul(mk)) {
    ScaffoldMessenger.of(context).showSnackBar(
      const SnackBar(content: Text('Kuota SKS penuh')),
    );
  }
},
```

Alasannya: controller yang bebas dari widget bisa di-**unit test** dengan Dart biasa, tanpa memompa widget dan tanpa emulator (dibahas di Modul 15). Selain itu, satu controller bisa dipakai beberapa View yang cara menampilkan pesannya berbeda.

> ℹ️ **Bagaimana dengan Get.snackbar?**
>
> GetX sebenarnya mengizinkan `Get.snackbar(...)`, `Get.dialog(...)`, dan `Get.to(...)` dipanggil langsung dari controller karena tidak butuh `BuildContext`. Kursus ini **sengaja tidak** melakukannya agar pemisahan tanggung jawab tetap jelas dan controller mudah diuji. Pemanggilan seperti itu kita letakkan di View.

### Instalasi

```
flutter pub add get
```

Lalu impor di file yang membutuhkannya:

```
import 'package:get/get.dart';
```

### GetMaterialApp

Ganti `MaterialApp` dengan `GetMaterialApp`. Parameternya sama (`home`, `theme`, `debugShowCheckedModeBanner`, dan seterusnya), ditambah fitur GetX seperti `initialBinding` dan `getPages`. `GetMaterialApp` wajib dipakai jika Anda memakai navigasi GetX (`Get.to`, `Get.back`) atau `Get.snackbar`.

```
return GetMaterialApp(
  debugShowCheckedModeBanner: false,
  theme: ThemeData(colorSchemeSeed: Colors.teal),
  home: const CounterView(),
);
```

### Contoh lengkap: counter dengan pola MVC

Agar mudah dicoba, Model, Controller, dan View ditulis dalam satu file dan dipisahkan dengan komentar. Di section 6 kita memecahnya menjadi beberapa file sesuai folder MVC. Jangan khawatir dulu tentang `.obs` dan `Obx`; keduanya dibahas tuntas di section 3.

**`lib/main.dart`**

```
// lib/main.dart — salin SELURUH kode ini untuk menggantikan isi lib/main.dart
// Pastikan sudah: flutter pub add get
import 'package:flutter/material.dart';
import 'package:get/get.dart';

// ================== MODEL ==================
// Data murni: tidak mengimpor Flutter maupun GetX.
class Hitungan {
  final int nilai;
  final int batas;

  const Hitungan({this.nilai = 0, this.batas = 10});

  bool get penuh => nilai >= batas;

  Hitungan tambahSatu() => Hitungan(nilai: nilai + 1, batas: batas);
}

// ================ CONTROLLER ================
// State + logika. Tidak memakai widget, BuildContext, atau SnackBar.
class CounterController extends GetxController {
  final hitungan = const Hitungan().obs; // state reaktif (dibahas di section 3)

  /// Mengembalikan false jika batas sudah tercapai. View yang memutuskan
  /// bagaimana memberi tahu pengguna.
  bool tambah() {
    if (hitungan.value.penuh) return false;
    hitungan.value = hitungan.value.tambahSatu();
    return true;
```

```
  }

  void reset() => hitungan.value = const Hitungan();
}

// ================== VIEW ==================
void main() {
  Get.put(CounterController()); // daftarkan controller sekali
  runApp(const MyApp());
}

class MyApp extends StatelessWidget {
  const MyApp({super.key});

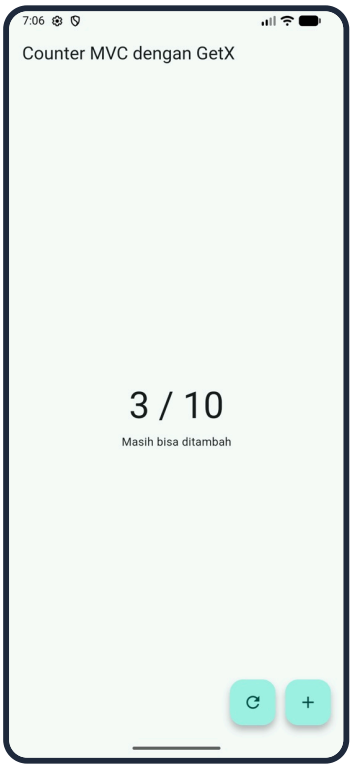
  @override
  Widget build(BuildContext context) {
    // GetMaterialApp = MaterialApp + fitur GetX (navigasi, binding, dll)
    return GetMaterialApp(
      debugShowCheckedModeBanner: false,
      theme: ThemeData(colorSchemeSeed: Colors.teal),
      home: const CounterView(),
    );
  }
}

class CounterView extends StatelessWidget {
  const CounterView({super.key});

  @override
  Widget build(BuildContext context) {
    final c = Get.find<CounterController>(); // ambil controller

    return Scaffold(
      appBar: AppBar(title: const Text('Counter MVC dengan GetX')),
      body: Center(
        // Obx: bagian ini dibangun ulang otomatis saat hitungan berubah
        child: Obx(() {
          final h = c.hitungan.value;
          return Column(
            mainAxisSize: MainAxisSize.min,
            children: [
              Text(
                '${h.nilai} / ${h.batas}',
                style: Theme.of(context).textTheme.displayMedium,
              ),
              const SizedBox(height: 8),
              Text(h.penuh ? 'Batas tercapai' : 'Masih bisa ditambah'),
            ],
          );
        }),
      ),
      floatingActionButton: Row(
        mainAxisSize: MainAxisSize.min,
        children: [
          FloatingActionButton(
            heroTag: 'reset',
            tooltip: 'Reset',
            onPressed: c.reset,
            child: const Icon(Icons.refresh),
```

```
          ),
          const SizedBox(width: 12),
          FloatingActionButton(
            heroTag: 'tambah',
            tooltip: 'Tambah',
            onPressed: () {
              final berhasil = c.tambah();
              if (!berhasil) {
                // Urusan tampilan (SnackBar) tetap di View
                ScaffoldMessenger.of(context).showSnackBar(
                  const SnackBar(content: Text('Sudah mencapai batas 10')),
                );
              }
            },
            child: const Icon(Icons.add),
          ),
        ],
      ),
    );
  }
}
```




Setelah tiga kali Tambah, tanpa setState

**Penjelasan:**

- **Model** `Hitungan` hanya berisi data dan aturan kecil (`penuh`, `tambahSatu`). Ia tidak tahu apa-apa tentang Flutter.
- **Controller** `CounterController` menyimpan state `hitungan` dan memutuskan boleh/tidaknya menambah. Method `tambah()` mengembalikan `bool`, bukan menampilkan SnackBar.

- **View** `CounterView` adalah `StatelessWidget`. Ia mengambil controller dengan `Get.find`, menampilkan state di dalam `Obx`, dan menampilkan SnackBar saat `tambah()` mengembalikan `false`.
- `Get.put(CounterController())` di `main()` mendaftarkan controller sekali sebelum aplikasi berjalan. Cara lain mendaftarkan controller dibahas di section 4.

> ⚠️ **Kesalahan umum**
>
> - Tetap memakai `MaterialApp` lalu memanggil `Get.to` atau `Get.snackbar`. Navigasi dan snackbar GetX butuh `GetMaterialApp`.
> - Mengimpor `material.dart` di controller supaya bisa memanggil `showSnackBar` atau menyimpan `BuildContext`. Kembalikan hasil saja, biarkan View yang menampilkan.
> - Menaruh logika bisnis di `onPressed` View (misalnya cek batas di tombol). Jika ada dua tombol di dua halaman, aturannya harus ditulis dua kali.

## 3. GetxController, .obs & Obx: State Reaktif dari Nol

State reaktif adalah inti GetX: nilai yang dibungkus `.obs` akan "mengabari" siapa pun yang membacanya ketika berubah, dan widget `Obx` otomatis membangun ulang dirinya. Tidak ada `setState`, tidak ada panggilan "beri tahu" manual.

### Tiga bahan utama

1. **`GetxController`**: class tempat state dan logika disimpan (huruf C pada MVC).
2. **`.obs`**: mengubah nilai biasa menjadi nilai yang bisa diamati (*observable*). `0.obs` menghasilkan `RxInt`, `'Andi'.obs` menghasilkan `RxString`, dan seterusnya.
3. **`Obx(() => ...)`**: widget yang membangun ulang isinya setiap kali observable yang *dibaca di dalamnya* berubah.

### Jenis observable

| Ditulis | Tipe | Membaca / mengubah |
|---|---|---|
| `0.obs` | `RxInt` | `x.value`, `x.value++` |
| `'Andi'.obs` | `RxString` | `x.value = 'Bunga'` |
| `false.obs` | `RxBool` | `x.value = true`, `x.toggle()` |
| `<String>[].obs` | `RxList<String>` | Langsung seperti list: `x.add(...)`, `x.remove(...)`, `x.length`, `x.assignAll(...)` |
| `RxnString()`, `Rxn<User>()` | Nullable (`String?`, `User?`) | `x.value = null` diperbolehkan |
| `const Hitungan().obs` | `Rx<Hitungan>` | Ganti objeknya: `x.value = Hitungan(...)` |

Kunci yang sering terlupa: nilai aslinya ada di **`.value`**. `RxList` adalah pengecualian yang nyaman, karena method list (`add`, `remove`, `length`, `[]`) bisa dipanggil langsung dan otomatis memberi tahu pendengarnya.

### Contoh kecil: tanpa UI sama sekali

Sebelum menyentuh widget, lihat bahwa observable sudah bisa "didengarkan". Worker `ever` menjalankan callback setiap kali nilainya berubah. Jalankan dengan `flutter run`, lalu lihat tulisan di Debug Console (layar emulator akan kosong karena tidak ada `runApp`).

```dart
// lib/main.dart — contoh tanpa UI, hasilnya muncul di konsol (Debug Console)
import 'package:flutter/foundation.dart';
import 'package:get/get.dart';

class CounterController extends GetxController {
  final nilai = 0.obs; // RxInt: int yang bisa "diamati"

  void tambah() => nilai.value++;
}

void main() {
  final c = CounterController();

  // ever: jalankan callback SETIAP kali nilai berubah
  ever<int>(c.nilai, (baru) => debugPrint('Nilai berubah menjadi $baru'));

  c.tambah(); // Nilai berubah menjadi 1
  c.tambah(); // Nilai berubah menjadi 2
  c.nilai.value = 2; // TIDAK tercetak: nilainya sama dengan sebelumnya
  debugPrint('Nilai akhir: ${c.nilai.value}'); // Nilai akhir: 2
}
```

- `0.obs` membuat `RxInt`. Nilainya dibaca dan diubah lewat `nilai.value`.
- `ever(observable, callback)` mendaftarkan fungsi yang dipanggil setiap kali nilai berubah. Kita akan memakai worker seperti ini lagi di section 9.
- Mengisi nilai yang **sama** (`2` menjadi `2`) tidak memicu callback. GetX hanya memberi tahu jika nilainya benar-benar berubah, sehingga rebuild yang sia-sia dihindari.
- Tidak ada pemanggilan "beri tahu pendengar" secara manual. Mengubah `.value` sudah cukup.

### Contoh lengkap: absensi kelas dengan Obx

Contoh ini memakai kelima jenis observable sekaligus: `RxString`, `RxInt`, `RxBool`, `RxList`, dan `RxnString`.

**`lib/main.dart`**

```dart
// lib/main.dart — salin SELURUH kode ini untuk menggantikan isi lib/main.dart
// Pastikan sudah: flutter pub add get
import 'package:flutter/material.dart';
import 'package:get/get.dart';

class AbsensiController extends GetxController {
  final kelas = 'Pemrograman Mobile A'.obs; // RxString
  final pertemuan = 1.obs; // RxInt
  final terkunci = false.obs; // RxBool
  final hadir = <String>[].obs; // RxList<String>
  final terakhir = RxnString(); // Rxn: boleh bernilai null

  final mahasiswa = const ['Andi', 'Bunga', 'Citra', 'Dodi', 'Eka'];

  void toggleHadir(String nama) {
    if (terkunci.value) return; // absensi dikunci, abaikan
```

```
    if (hadir.contains(nama)) {
      hadir.remove(nama); // RxList memberi tahu Obx otomatis
    } else {
      hadir.add(nama);
    }
    terakhir.value = nama;
  }

  void toggleKunci() => terkunci.toggle(); // sama dengan value = !value

  void pertemuanBerikutnya() {
    pertemuan.value++;
    hadir.clear();
    terakhir.value = null;
    terkunci.value = false;
  }
}

void main() {
  Get.put(AbsensiController());
  runApp(const MyApp());
}

class MyApp extends StatelessWidget {
  const MyApp({super.key});

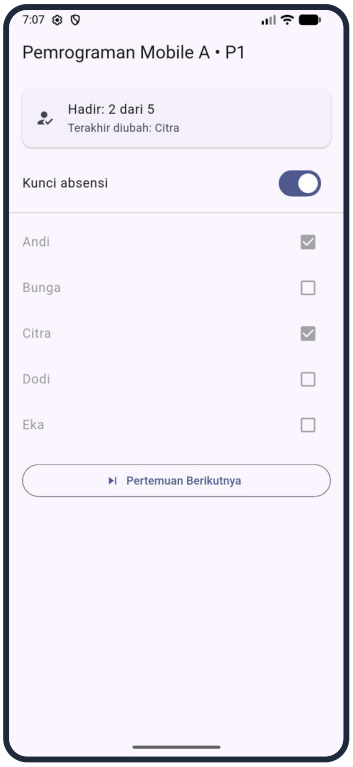
  @override
  Widget build(BuildContext context) {
    return GetMaterialApp(
      debugShowCheckedModeBanner: false,
      theme: ThemeData(colorSchemeSeed: Colors.indigo),
      home: const AbsensiView(),
    );
  }
}

// StatelessWidget: tidak ada setState sama sekali
class AbsensiView extends StatelessWidget {
  const AbsensiView({super.key});

  @override
  Widget build(BuildContext context) {
    final c = Get.find<AbsensiController>();

    return Scaffold(
      appBar: AppBar(
        // Obx kecil 1: hanya membaca kelas dan pertemuan
        title: Obx(() => Text('${c.kelas.value} • P${c.pertemuan.value}')),
      ),
      body: ListView(
        children: [
          // Obx kecil 2: ringkasan
          Obx(
            () => Card(
              margin: const EdgeInsets.all(16),
              child: ListTile(
                leading: const Icon(Icons.how_to_reg),
                title: Text('Hadir: ${c.hadir.length} dari ${c.mahasiswa.length}'),
                subtitle: Text('Terakhir diubah: ${c.terakhir.value ?? '-'}'),
```

```
              ),
            ),
          ),
          // Obx kecil 3: saklar kunci
          Obx(
            () => SwitchListTile(
              title: const Text('Kunci absensi'),
              value: c.terkunci.value,
              onChanged: (_) => c.toggleKunci(),
            ),
          ),
          const Divider(),
          // Satu Obx per baris: baris lain tidak ikut dibangun ulang
          for (final nama in c.mahasiswa)
            Obx(
              () => CheckboxListTile(
                title: Text(nama),
                value: c.hadir.contains(nama),
                onChanged: c.terkunci.value ? null : (_) => c.toggleHadir(nama),
              ),
            ),
          Padding(
            padding: const EdgeInsets.all(16),
            child: OutlinedButton.icon(
              onPressed: c.pertemuanBerikutnya,
              icon: const Icon(Icons.skip_next),
              label: const Text('Pertemuan Berikutnya'),
            ),
          ),
        ],
      ),
    );
  }
}
```




Andi dan Citra dicentang, lalu absensi dikunci

**Penjelasan:**

- `AbsensiView` adalah `StatelessWidget`, tetapi ringkasan, saklar, dan checkbox tetap berubah. Yang memicu rebuild adalah `Obx`, bukan `setState`.
- Setiap `Obx` hanya mendengarkan observable yang **dibaca di dalamnya**. Obx di `AppBar` membaca `kelas` dan `pertemuan`, sehingga tidak ikut dibangun ulang saat checkbox dicentang.
- Setiap baris mahasiswa punya `Obx` sendiri. Baris itu membaca `hadir` dan `terkunci`, jadi ketika kunci diaktifkan semua checkbox langsung menjadi abu-abu (nonaktif).
- `terakhir` bertipe `RxnString` karena awalnya belum ada nama yang diubah (`null`). Di View kita tampilkan `'–'` dengan operator `??`.
- `hadir.clear()`, `hadir.add()`, dan `hadir.remove()` langsung memicu Obx karena `hadir` adalah `RxList`.

### Bedanya dengan setState

| Aspek | `setState` | GetX ( `.obs` + `Obx` ) |
|---|---|---|
| Letak state | Di dalam `State` milik satu widget | Di controller, terpisah dari widget |
| Siapa yang bisa membaca | Widget itu sendiri (dan anak yang dititipi lewat constructor) | Widget mana pun di halaman mana pun (lewat `Get.find` ) |
| Yang dibangun ulang | Seluruh method `build` widget tersebut | Hanya isi `Obx` yang membaca observable yang berubah |
| Cara memicu | Memanggil `setState(() {...})` secara manual | Otomatis saat `.value` (atau isi `RxList` ) berubah |
| Cocok untuk | State lokal: isi TextField, tab aktif, animasi | State bersama: keranjang, KRS, user login |

⚠️ **Kesalahan umum**

- **Lupa `.value`.** `Text('${c.pertemuan}')` memang menampilkan angka, tetapi biasakan menulis `c.pertemuan.value` agar jelas yang dibaca adalah isinya. Saat mengubah, `c.pertemuan = 2` bahkan error; yang benar `c.pertemuan.value = 2` .
- **Obx yang tidak membaca observable apa pun.** Misalnya `Obx(() => Text(namaBiasa))` . GetX akan melempar error "improper use of GetX" karena Obx tidak tahu harus mendengarkan apa.
- **Membaca `.value` di luar Obx**, misalnya `final n = c.pertemuan.value;` di awal `build` lalu dipakai di `Text` biasa. Nilai itu dibaca sekali dan tidak pernah diperbarui.
- Membuat `.obs` di dalam method `build` . Setiap rebuild membuat observable baru dan state hilang. Simpan observable di controller.

## 4. Dependency Injection: Get.put, Get.find & Get.lazyPut

**Dependency injection** berarti sebuah objek (misalnya controller) dibuat dan disediakan oleh pihak luar, bukan dibuat sendiri oleh widget yang membutuhkannya. Di GetX, kita "menitipkan" controller ke sebuah wadah global dengan `Get.put`, lalu widget mana pun bisa mengambilnya dengan `Get.find`, tanpa constructor berantai.

### Daftar API yang perlu dikenal

| API | Fungsinya | Kapan dipakai |
|---|---|---|
| `Get.put(X())` | Membuat objek sekarang juga lalu mendaftarkannya | Controller yang langsung dibutuhkan |
| `Get.put(X(), permanent: true)` | Seperti di atas, tetapi tidak pernah dihapus otomatis | Controller global (auth, tema, keranjang) |
| `Get.lazyPut(() => X())` | Mendaftarkan "resep"; objek baru dibuat saat pertama kali di-`find` | Controller yang belum tentu dipakai |
| `Get.lazyPut(..., fenix: true)` | Resep tetap disimpan; jika objek pernah dihapus, ia dibuat ulang saat di-`find` lagi | Halaman yang bisa dibuka-tutup berkali-kali |
| `Get.find<X>()` | Mengambil objek yang sudah terdaftar | Di View atau di controller lain |
| `Get.delete<X>()` | Menghapus objek dan memanggil `onClose()`-nya | Jarang dipanggil manual; GetX biasanya mengurusnya |
| `GetView<X>` | `StatelessWidget` yang sudah punya getter `controller` | View yang memakai satu controller utama |

### Contoh kecil: mendaftarkan dan mengambil

```dart
import 'package:flutter/material.dart';
import 'package:get/get.dart';

class CounterController extends GetxController {
  final nilai = 0.obs;
}

class ProfilController extends GetxController {}

class KeranjangController extends GetxController {}

void daftarkan() {
  // 1) Get.put: objek dibuat SEKARANG lalu didaftarkan
```

```
  Get.put(CounterController());

  // 2) permanent: true → tidak pernah dihapus otomatis oleh GetX
  Get.put(ProfilController(), permanent: true);

  // 3) Get.lazyPut: yang didaftarkan "resep"-nya; objek baru dibuat
  //    saat pertama kali di-Get.find. fenix: true → dibuat ulang jika
  //    sebelumnya sudah dihapus.
  Get.lazyPut(() => KeranjangController(), fenix: true);
}

void ambil() {
  // 4) Get.find: ambil objek yang sudah terdaftar (error jika belum)
  final c = Get.find<CounterController>();
  c.nilai.value++;
}

Future<void> hapus() async {
  // 5) Get.delete: hapus dari memori, onClose() milik controller dipanggil
  await Get.delete<CounterController>();
}

// 6) GetView<T>: StatelessWidget dengan getter `controller` = Get.find<T>()
class NilaiView extends GetView<CounterController> {
  const NilaiView({super.key});

  @override
  Widget build(BuildContext context) {
    return Obx(() => Text('Nilai: ${controller.nilai.value}'));
  }
}
```

- Tipe generik di `Get.find<CounterController>()` wajib ditulis. Tipe itulah "kunci" pencariannya.
- `Get.find` sebelum ada `Get.put` / `Get.lazyPut` akan error dengan pesan *"CounterController" not found. You need to call "Get.put(CounterController())" or "Get.lazyPut(()⇒CounterController())"*.
- `GetView` hanyalah jalan pintas: di dalamnya `controller` sama dengan `Get.find<CounterController>()`. Constructor-nya tetap boleh `const`.

> ℹ️ **Kapan controller dihapus otomatis?**
>
> Secara bawaan GetX menghapus controller yang dibuat *ketika sebuah halaman dibuka* (misalnya `Get.put` di dalam View halaman itu atau lewat binding halaman) begitu halaman tersebut ditutup dengan `Get.back`. Controller yang didaftarkan di `main()` sebelum `runApp`, terutama dengan `permanent: true`, tetap hidup selama aplikasi berjalan. Detail siklus hidup ini dibahas di section 7.

### Contoh lengkap: satu controller, dua halaman

Halaman Dua dibuka dengan `Get.to` tanpa mengirim data apa pun, tetapi bisa membaca dan mengubah nilai yang sama dengan Halaman Satu.

**lib/main.dart**

```
// lib/main.dart — salin SELURUH kode ini untuk menggantikan isi lib/main.dart
// Pastikan sudah: flutter pub add get
import 'package:flutter/material.dart';
import 'package:get/get.dart';

class CounterController extends GetxController {
  final nilai = 0.obs;

  void tambah() => nilai.value++;

  void kurang() {
    if (nilai.value == 0) return;
    nilai.value--;
  }
}

void main() {
  // Didaftarkan SEKALI untuk seluruh aplikasi
  Get.put(CounterController(), permanent: true);
  runApp(const MyApp());
}

class MyApp extends StatelessWidget {
  const MyApp({super.key});

  @override
  Widget build(BuildContext context) {
    return GetMaterialApp(
      debugShowCheckedModeBanner: false,
      theme: ThemeData(colorSchemeSeed: Colors.indigo),
      home: const HalamanSatuView(),
    );
  }
}

// GetView<T> menyediakan getter `controller` (= Get.find<T>())
class HalamanSatuView extends GetView<CounterController> {
  const HalamanSatuView({super.key});

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(title: const Text('Halaman Satu')),
      body: Center(
        child: Column(
          mainAxisSize: MainAxisSize.min,
          children: [
            Obx(
              () => Text(
                'Nilai: ${controller.nilai.value}',
                style: Theme.of(context).textTheme.headlineMedium,
```

```
            ),
          ),
          const SizedBox(height: 16),
          FilledButton(
            onPressed: controller.tambah,
            child: const Text('Tambah'),
          ),
          const SizedBox(height: 8),
          OutlinedButton(
            // Tidak ada data yang dikirim lewat constructor!
            onPressed: () => Get.to(() => const HalamanDuaView()),
            child: const Text('Buka Halaman Dua'),
          ),
        ],
      ),
    ),
  );
 }
}

class HalamanDuaView extends GetView<CounterController> {
  const HalamanDuaView({super.key});

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(title: const Text('Halaman Dua')),
      body: Center(
        child: Column(
          mainAxisSize: MainAxisSize.min,
          children: [
            Obx(
              () => Text(
                'Nilai yang sama: ${controller.nilai.value}',
                style: Theme.of(context).textTheme.headlineMedium,
              ),
            ),
            const SizedBox(height: 16),
            FilledButton.tonal(
              onPressed: controller.kurang,
              child: const Text('Kurangi dari Halaman Dua'),
            ),
            const SizedBox(height: 8),
            TextButton(
              onPressed: () => Get.back(),
              child: const Text('Kembali'),
            ),
          ],
        ),
      ),
    );
  }
}
```




Tiga kali Tambah di Halaman Satu, lalu satu kali Kurangi di Halaman Dua

**Penjelasan:**

- `Get.put(CounterController(), permanent: true)` di `main()` membuat **satu** objek untuk seluruh aplikasi. Kedua halaman memakai objek yang sama, sehingga nilainya selalu sinkron.
- Kedua View `extends GetView<CounterController>`, jadi cukup menulis `controller.nilai` tanpa `Get.find`.
- `Get.to(() => const HalamanDuaView())` membuka halaman baru tanpa `BuildContext`, dan `Get.back()` menutupnya. Keduanya butuh `GetMaterialApp`.
- Saat menekan "Kurangi" di Halaman Dua lalu kembali, Halaman Satu sudah menampilkan nilai terbaru karena `Obx` di sana juga mendengarkan `nilai` yang sama.
- Tombol "Tambah" langsung memakai `controller.tambah` (tanpa `Obx`). Memanggil method tidak perlu berlangganan perubahan; hanya bagian yang *menampilkan* state yang dibungkus `Obx`.

**Alternatif: lazyPut**

Jika controller belum tentu dipakai, ganti baris pendaftaran di `main()` menjadi:

```
Get.lazyPut(() => CounterController(), fenix: true);
```

Objek baru dibuat ketika `controller` pertama kali dibaca oleh `HalamanSatuView`. Dengan `fenix: true`, jika suatu saat objek itu dihapus, GetX membuatnya lagi saat dibutuhkan (nilainya kembali ke awal).

⚠️ **Kesalahan umum**

- `Get.find` **sebelum** `Get.put`**.** Muncul error *"... not found. You need to call "Get.put(...)" or "Get.lazyPut(...)"*. Pastikan controller didaftarkan di `main()`, di `initialBinding`, atau di binding halaman sebelum View membacanya.
- `Get.put` **di dalam method** `build` **berkali-kali** dengan maksud membuat controller "baru". `Get.put` untuk tipe yang sudah terdaftar mengembalikan objek lama. Daftarkan sekali di tempat yang jelas.
- Membuat objek controller sendiri di View, misalnya `final c = CounterController();`. Objek itu tidak terdaftar, sehingga halaman lain yang memakai `Get.find` mendapat objek yang berbeda dan nilainya tidak sinkron.
- Lupa tipe generik: `Get.find()` tanpa `<CounterController>` (dan tanpa tipe variabel) membuat GetX tidak tahu objek mana yang dicari.

## 5. Obx, GetX & GetBuilder: Mengatur Rebuild

GetX menyediakan tiga widget untuk menampilkan state controller: `Obx`, `GetX<T>`, dan `GetBuilder<T>`. Perbedaannya ada pada **apa yang memicu rebuild**. Memilih yang tepat membuat aplikasi benar sekaligus hemat rebuild.

### Tiga cara menampilkan state

```dart
import 'package:flutter/material.dart';
import 'package:get/get.dart';

class ProfilController extends GetxController {
  final nama = 'Andi'.obs; // reaktif
  int bintang = 0; // biasa (non-reaktif)

  void tambahBintang() {
    bintang++;
    update(); // wajib untuk GetBuilder
  }
}

Widget contoh() {
  final c = Get.find<ProfilController>();
  return Column(
    children: [
      // 1) Obx: ambil controller sendiri, rebuild saat .obs yang dibaca berubah
      Obx(() => Text('Nama: ${c.nama.value}')),

      // 2) GetX<T>: sama-sama reaktif, controller diberikan lewat builder
      GetX<ProfilController>(
        builder: (ctrl) => Text('Nama: ${ctrl.nama.value}'),
      ),

      // 3) GetBuilder<T>: tidak memantau .obs, rebuild hanya saat update()
      GetBuilder<ProfilController>(
        builder: (ctrl) => Text('Bintang: ${ctrl.bintang}'),
      ),
    ],
  );
}
```

| Widget | Rebuild ketika | Cara mendapat controller | State yang cocok |
|---|---|---|---|
| `Obx(() => ...)` | Observable (`.obs`) yang *dibaca di dalamnya* berubah | Sendiri: `Get.find` atau `controller` dari `GetView` | Reaktif (`.obs`) |
| `GetX<T>(builder: ...)` | Sama seperti Obx | Diberikan lewat parameter `builder`; bisa juga `init:` untuk membuatnya | Reaktif (`.obs`) |
| `GetBuilder<T>(builder: ...)` | Controller memanggil `update()` (atau `update([id])`) | Diberikan lewat parameter `builder` | Field biasa (non-reaktif) |

Hal penting: `GetBuilder` **tidak** memantau `.obs`, dan `Obx` / `GetX` **tidak** peduli pada `update()`. Keduanya adalah dua "jalur" yang terpisah.

### Contoh lengkap: membuktikan perbedaan rebuild

Setiap kartu menampilkan berapa kali ia di-build. Tekan tombol dan amati angka mana yang naik.

`lib/main.dart`

```dart
// lib/main.dart — salin SELURUH kode ini untuk menggantikan isi lib/main.dart
// Demo rebuild: angka "build ke-N" menunjukkan berapa kali tiap bagian di-build
import 'package:flutter/material.dart';
import 'package:get/get.dart';

class ProfilController extends GetxController {
  // Reaktif: dipantau oleh Obx / GetX
  final nama = 'Andi'.obs;
  final poin = 0.obs;

  // Non-reaktif: hanya diperbarui lewat update() untuk GetBuilder
  int bintang = 0;

  void tambahPoin() => poin.value++;

  void gantiNama() => nama.value = nama.value == 'Andi' ? 'Bunga' : 'Andi';

  void tambahBintang() {
    bintang++;
    update(); // beri tahu semua GetBuilder<ProfilController>
  }
}

// Penghitung build HANYA untuk keperluan demo (jangan ditiru di aplikasi nyata)
final Map<String, int> jumlahBuild = {};
int catatBuild(String nama) => jumlahBuild[nama] = (jumlahBuild[nama] ?? 0) + 1;

void main() {
  Get.put(ProfilController());
```

```dart
  runApp(const MyApp());
}

class MyApp extends StatelessWidget {
  const MyApp({super.key});

  @override
  Widget build(BuildContext context) {
    return GetMaterialApp(
      debugShowCheckedModeBanner: false,
      theme: ThemeData(colorSchemeSeed: Colors.deepOrange),
      home: const DemoView(),
    );
  }
}

class DemoView extends GetView<ProfilController> {
  const DemoView({super.key});

  @override
  Widget build(BuildContext context) {
    // Halaman ini tidak membaca .value di luar Obx, jadi tidak ikut rebuild
    final n = catatBuild('halaman');
    return Scaffold(
      appBar: AppBar(title: Text('Obx vs GetX vs GetBuilder ($n)')),
      body: ListView(
        padding: const EdgeInsets.all(12),
        children: [
          const KartuObxSemua(),
          const KartuObxNama(),
          const KartuGetX(),
          const KartuGetBuilder(),
          const SizedBox(height: 12),
          Row(
            children: [
              Expanded(
                child: FilledButton(
                  onPressed: controller.tambahPoin,
                  child: const Text('Tambah Poin'),
                ),
              ),
              const SizedBox(width: 8),
              Expanded(
                child: OutlinedButton(
                  onPressed: controller.gantiNama,
                  child: const Text('Ganti Nama'),
                ),
              ),
            ],
          ),
          const SizedBox(height: 8),
          FilledButton.tonal(
            onPressed: controller.tambahBintang,
            child: const Text('Tambah Bintang'),
          ),
        ],
      ),
    );
  }
```

```dart
}

// 1) Obx yang membaca nama DAN poin: rebuild jika salah satunya berubah
class KartuObxSemua extends GetView<ProfilController> {
  const KartuObxSemua({super.key});

  @override
  Widget build(BuildContext context) {
    return Card(
      child: Obx(() {
        final n = catatBuild('obx-semua');
        return ListTile(
          title: Text('Obx: ${controller.nama.value}, ${controller.poin.value} poin'),
          subtitle: Text('build ke-$n'),
        );
      }),
    );
  }
}

// 2) Obx yang hanya membaca nama: rebuild HANYA jika nama berubah
class KartuObxNama extends GetView<ProfilController> {
  const KartuObxNama({super.key});

  @override
  Widget build(BuildContext context) {
    return Card(
      child: Obx(() {
        final n = catatBuild('obx-nama');
        return ListTile(
          title: Text('Obx(nama): ${controller.nama.value}'),
          subtitle: Text('build ke-$n'),
        );
      }),
    );
  }
}

// 3) GetX<T>: seperti Obx, tetapi controller diberikan lewat parameter builder
class KartuGetX extends StatelessWidget {
  const KartuGetX({super.key});

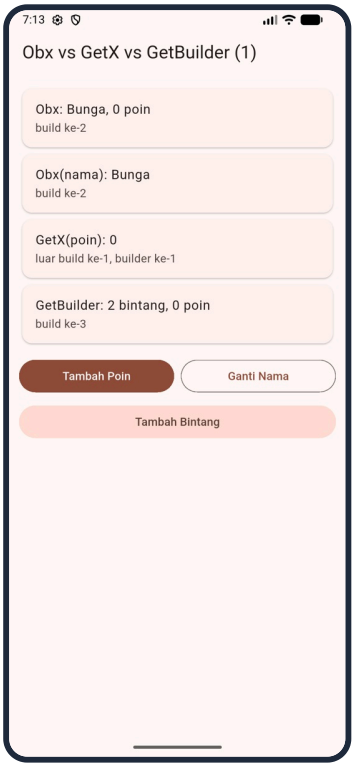
  @override
  Widget build(BuildContext context) {
    final nLuar = catatBuild('getx-luar');
    return Card(
      child: GetX<ProfilController>(
        builder: (c) {
          final nDalam = catatBuild('getx-dalam');
          return ListTile(
            title: Text('GetX(poin): ${c.poin.value}'),
            subtitle: Text('luar build ke-$nLuar, builder ke-$nDalam'),
          );
        },
      ),
    );
  }
}
```

```dart
// 4) GetBuilder: TIDAK memantau .obs, hanya bereaksi pada update()
class KartuGetBuilder extends StatelessWidget {
  const KartuGetBuilder({super.key});

  @override
  Widget build(BuildContext context) {
    return Card(
      child: GetBuilder<ProfilController>(
        builder: (c) {
          final n = catatBuild('getbuilder');
          return ListTile(
            title: Text('GetBuilder: ${c.bintang} bintang, ${c.poin.value} poin'),
            subtitle: Text('build ke-$n'),
          );
        },
      ),
    );
  }
}
```




Setelah 3x Tambah Poin: Obx(nama) dan GetBuilder tidak rebuild




Dari awal: 1x Ganti Nama lalu 2x Tambah Bintang

**Penjelasan hasil:**

- **Tambah Poin** (gambar kiri): kartu *Obx* (membaca nama dan poin) serta *GetX(poin)* naik menjadi build ke-4. Kartu *Obx(nama)* tetap build ke-1 karena ia tidak membaca `poin`.
- Kartu *GetBuilder* juga tetap build ke-1, bahkan menampilkan "0 poin" padahal poin sudah 3. Mengubah `.obs` tidak memicu GetBuilder, dan membaca `.value` di dalam GetBuilder tidak membuatnya reaktif.
- **Ganti Nama** lalu **Tambah Bintang** (gambar kanan): Ganti Nama menaikkan kedua kartu Obx. Dua kali `update()` menaikkan GetBuilder menjadi build ke-3, sedangkan kartu GetX tetap

build ke-1.

- Angka "luar" pada kartu GetX tidak pernah naik. Seperti Obx, hanya isi `builder` yang dibangun ulang.
- Angka di judul AppBar tetap (1): `DemoView` sendiri tidak pernah rebuild, karena tombol hanya *memanggil* method controller.

**Membatasi rebuild**

Prinsipnya sama untuk ketiga widget: **bungkus sekecil mungkin** bagian yang benar-benar berubah.

1. Jangan membungkus satu halaman penuh dengan satu `Obx` besar. Pecah menjadi beberapa `Obx` kecil, misalnya satu untuk judul, satu untuk ringkasan, satu per baris list.
2. Di dalam Obx, baca hanya observable yang dibutuhkan. Obx yang membaca `nama` saja tidak akan rebuild saat `poin` berubah.
3. Di callback (`onPressed`, `onTap`) cukup panggil method controller. Tidak perlu Obx.
4. Untuk `GetBuilder`, pakai `id` agar `update` hanya membangun ulang bagian tertentu.

**Opsional: GetBuilder dengan id**

```
import 'package:flutter/material.dart';
import 'package:get/get.dart';

class NilaiController extends GetxController {
  int uts = 0;
  int uas = 0;

  void tambahUts() {
    uts++;
    update(['uts']); // hanya GetBuilder dengan id 'uts' yang rebuild
  }

  void tambahUas() {
    uas++;
    update(['uas']);
  }
}

Widget kartuNilai() {
  return Column(
    children: [
      GetBuilder<NilaiController>(
        id: 'uts',
        builder: (c) => Text('UTS: ${c.uts}'),
      ),
      GetBuilder<NilaiController>(
        id: 'uas',
        builder: (c) => Text('UAS: ${c.uas}'),
      ),
    ],
  );
}
```

`update(['uts'])` hanya membangun ulang GetBuilder dengan `id: 'uts'`. Perhatikan juga kebalikannya: `update()` tanpa argumen hanya membangun ulang GetBuilder yang **tidak** memakai `id`.

### Ringkasan & tips memilih

| Situasi | Pilihan |
|---|---|
| Menampilkan state `.obs` (pilihan utama di kursus ini) | `Obx` |
| Butuh controller langsung di builder, atau ingin membuatnya di tempat dengan `init:` | `GetX<T>` |
| State berupa field biasa dan Anda ingin mengontrol sendiri kapan UI diperbarui | `GetBuilder<T>` + `update()` |
| Hanya memanggil method (tombol, gesture) | Tanpa widget apa pun: `controller.method()` |

> 💡 **Cara memilih**
>
> 1. Default-nya pakai `.obs` + `Obx`. Sederhana dan otomatis.
> 2. Pakai `GetBuilder` jika Anda sengaja tidak memakai `.obs`, misalnya data yang diubah berkali-kali lalu ditampilkan sekali di akhir proses.
> 3. Jangan mencampur dua jalur untuk satu data. Jika `poin` sudah `.obs`, tampilkan dengan Obx, bukan GetBuilder.

⚠️ **Kesalahan umum**

- **Lupa** `update()` setelah mengubah field biasa. Data berubah di memori, tetapi GetBuilder tidak pernah dibangun ulang.
- **Mengharapkan GetBuilder bereaksi pada** `.obs`, seperti kartu GetBuilder pada demo di atas yang tetap "0 poin".
- **Satu Obx untuk seluruh halaman.** Benar secara fungsi, tetapi setiap perubahan kecil membangun ulang semuanya. Pecah menjadi Obx kecil.
- Memanggil `update()` di dalam `builder`. Widget diminta rebuild saat sedang dibangun, sehingga muncul error atau rebuild berulang. `update()` hanya dipanggil di method yang mengubah state.

## 6. Menyusun MVC: Contoh KRS (Model, View, Controller)

Sekarang kita gabungkan semua yang sudah dipelajari dalam satu contoh nyata yang dipecah menjadi beberapa file sesuai pola MVC: pengisian KRS dengan batas maksimal 24 SKS.

### Struktur folder

```
lib/
├── main.dart                      # GetMaterialApp + pendaftaran controller
├── models/
|   └── mata_kuliah.dart           # M: bentuk data
├── controllers/
|   └── krs_controller.dart        # C: state reaktif + aturan 24 SKS
└── views/
    └── katalog_krs_view.dart      # V: katalog + ringkasan KRS
```

Pasang package terlebih dahulu jika belum: `flutter pub add get`. Lalu buat folder `models`, `controllers`, dan `views` di dalam `lib/`.

### Langkah 1: Model

**`lib/models/mata_kuliah.dart`**

```dart
// lib/models/mata_kuliah.dart
// MODEL: data murni, tanpa import Flutter maupun GetX
class MataKuliah {
  final String nama;
  final int sks;

  const MataKuliah(this.nama, this.sks);
}
```

Model hanya berisi data. Tidak ada `import` sama sekali, sehingga bisa dipakai di mana saja, termasuk di unit test.

### Langkah 2: Controller

**`lib/controllers/krs_controller.dart`**

```dart
// lib/controllers/krs_controller.dart
// CONTROLLER: state + aturan bisnis. Tidak mengimpor material.dart.
import 'package:get/get.dart';

import '../models/mata_kuliah.dart';

class KrsController extends GetxController {
  static const int batasMaksimalSks = 24;

  // Katalog (data statis). Di section 8 data seperti ini diambil lewat service.
  final List<MataKuliah> katalog = const [
    MataKuliah('Pemrograman Mobile', 4),
```

```
    MataKuliah('Basis Data Lanjut', 3),
    MataKuliah('Kecerdasan Buatan', 3),
    MataKuliah('Jaringan Komputer', 3),
    MataKuliah('Rekayasa Perangkat Lunak', 3),
    MataKuliah('Keamanan Siber', 3),
    MataKuliah('Etika Profesi', 2),
    MataKuliah('Kerja Praktik', 4),
  ];

  // State privat reaktif: hanya controller yang boleh mengubah
  final _matkulTerpilih = <MataKuliah>[].obs;

  // Getter publik read-only
  List<MataKuliah> get matkulTerpilih => List.unmodifiable(_matkulTerpilih);

  // Data turunan = getter, tidak disimpan terpisah
  int get totalSks => _matkulTerpilih.fold(0, (total, mk) => total + mk.sks);

  bool sudahDiambil(MataKuliah mk) => _matkulTerpilih.contains(mk);

  /// Mengembalikan false jika kuota terlampaui (atau matkul sudah diambil).
  /// Controller TIDAK menampilkan SnackBar; View yang memutuskan.
  bool tambahMatkul(MataKuliah mk) {
    if (sudahDiambil(mk)) return false;
    if (totalSks + mk.sks > batasMaksimalSks) return false;
    _matkulTerpilih.add(mk); // RxList otomatis memberi tahu Obx
    return true;
  }

  void hapusMatkul(int index) => _matkulTerpilih.removeAt(index);
}
```

- `_matkulTerpilih` adalah `RxList` **privat**. Dari luar file ini, satu-satunya cara mengubahnya adalah lewat `tambahMatkul` dan `hapusMatkul`, sehingga aturan 24 SKS tidak bisa dilewati.
- Getter `matkulTerpilih` mengembalikan `List.unmodifiable`. Jika View mencoba `controller.matkulTerpilih.add(...)`, aplikasi langsung error dan kesalahan ketahuan sejak awal.
- `totalSks` adalah **getter** (data turunan), bukan field terpisah. Nilainya selalu dihitung dari isi list, jadi tidak mungkin "tidak sinkron". Karena getter ini membaca `_matkulTerpilih`, Obx yang memanggil `controller.totalSks` otomatis ikut mendengarkan list tersebut.
- `tambahMatkul` mengembalikan `bool`. Controller yang memutuskan boleh/tidaknya (aturan bisnis), sedangkan View yang menampilkan pesan.
- Hanya `package:get/get.dart` yang diimpor, tanpa `material.dart`. Controller ini bisa diuji dengan unit test biasa.

### Langkah 3: View

**`lib/views/katalog_krs_view.dart`**

```
// lib/views/katalog_krs_view.dart
// VIEW: hanya menampilkan state dan meneruskan aksi ke controller
import 'package:flutter/material.dart';
```

```dart
import 'package:get/get.dart';

import '../controllers/krs_controller.dart';

class KatalogKrsView extends GetView<KrsController> {
  const KatalogKrsView({super.key});

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(title: const Text('Pengisian KRS')),
      body: Column(
        children: [
          const RingkasanKrs(),
          Expanded(
            child: ListView.builder(
              itemCount: controller.katalog.length,
              itemBuilder: (context, index) {
                final mk = controller.katalog[index];
                return ListTile(
                  title: Text(mk.nama),
                  subtitle: Text('${mk.sks} SKS'),
                  // Obx kecil per baris: ikon berubah jika matkul sudah diambil
                  trailing: Obx(() {
                    final diambil = controller.sudahDiambil(mk);
                    return IconButton(
                      tooltip: 'Ambil ${mk.nama}',
                      icon: Icon(
                        diambil ? Icons.check_circle : Icons.add_circle,
                        color: diambil ? Colors.green : Colors.indigo,
                      ),
                      onPressed: diambil
                          ? null
                          : () {
                              final berhasil = controller.tambahMatkul(mk);
                              if (!berhasil) {
                                ScaffoldMessenger.of(context).showSnackBar(
                                  const SnackBar(
                                    content: Text('Kuota maksimal 24 SKS terlampaui!'),
                                    backgroundColor: Colors.red,
                                  ),
                                );
                              }
                            },
                    );
                  }),
                );
              },
            ),
          ),
        ],
      ),
    );
  }
}

class RingkasanKrs extends GetView<KrsController> {
  const RingkasanKrs({super.key});
```

```
  @override
  Widget build(BuildContext context) {
    return Card(
      margin: const EdgeInsets.all(16),
      child: Padding(
        padding: const EdgeInsets.all(16),
        child: Obx(() {
          final total = controller.totalSks;
          const batas = KrsController.batasMaksimalSks;
          final terpilih = controller.matkulTerpilih;
          return Column(
            children: [
              Text('Total SKS Diambil: $total / $batas SKS'),
              const SizedBox(height: 8),
              LinearProgressIndicator(
                value: total / batas,
                color: total >= 20 ? Colors.orange : Colors.blue,
              ),
              const SizedBox(height: 8),
              Wrap(
                spacing: 6,
                children: [
                  for (var i = 0; i < terpilih.length; i++)
                    InputChip(
                      label: Text(terpilih[i].nama),
                      onDeleted: () => controller.hapusMatkul(i),
                    ),
                ],
              ),
            ],
          );
        }),
      ),
    );
  }
}
```

### Langkah 4: Mendaftarkan controller di main.dart

`lib/main.dart`

```
// lib/main.dart — salin SELURUH kode ini untuk menggantikan isi lib/main.dart
// Pastikan sudah: (1) flutter pub add get, (2) membuat ketiga file di atas
import 'package:flutter/material.dart';
import 'package:get/get.dart';

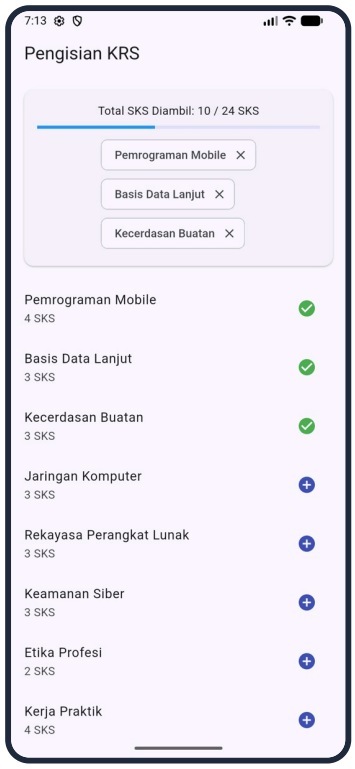
import 'controllers/krs_controller.dart';
import 'views/katalog_krs_view.dart';

void main() {
  runApp(const MyApp());
}

class MyApp extends StatelessWidget {
  const MyApp({super.key});

  @override
  Widget build(BuildContext context) {
```

```
    return GetMaterialApp(
      debugShowCheckedModeBanner: false,
      theme: ThemeData(colorSchemeSeed: Colors.indigo),
      // Dijalankan sekali sebelum halaman pertama dibangun
      initialBinding: BindingsBuilder(() {
        Get.put(KrsController());
      }),
      home: const KatalogKrsView(),
    );
  }
}
```




Setelah mengambil tiga mata kuliah: total 10 SKS, ikon berubah menjadi centang

`initialBinding` dijalankan sekali sebelum halaman pertama dibangun, sehingga `KrsController` sudah terdaftar ketika `KatalogKrsView` membaca `controller`. Cara lain yang sama-sama benar adalah mendaftarkannya di `main()`:

```
void main() {
  Get.put(KrsController(), permanent: true);
  runApp(const MyApp());
}
```

Pilih salah satu saja. Bindings yang lebih rapi (class `Bindings` sendiri dan binding per halaman) dibahas di section 7.

**Penjelasan:**

- `KatalogKrsView` tidak dibungkus Obx, sehingga daftar katalog tidak ikut dibangun ulang ketika SKS bertambah. Hanya ikon di setiap baris yang punya Obx kecil sendiri.

- `RingkasanKrs` membungkus isinya dengan satu `Obx`, sehingga total SKS, progress bar, dan chip selalu sinkron.
- `InputChip.onDeleted` memanggil `controller.hapusMatkul(i)`. Ikon di katalog otomatis kembali menjadi tombol tambah karena Obx barisnya membaca `sudahDiambil`.
- SnackBar ditampilkan di View dengan `ScaffoldMessenger` berdasarkan nilai `bool` dari controller. Controller tidak tahu apa-apa tentang SnackBar.
- `RingkasanKrs` tidak menerima data apa pun lewat constructor. Bandingkan dengan contoh prop drilling di section 1.

> ⚠️ **Kesalahan umum**
>
> - Memeriksa batas SKS di View, bukan di controller. Jika ada dua tombol tambah di dua halaman, aturannya harus ditulis dua kali.
> - Mengembalikan `_matkulTerpilih` langsung dari getter, atau membuatnya publik. View bisa memanggil `.add()` dan melewati pengecekan kuota.
> - Menyimpan `totalSks` sebagai `.obs` terpisah lalu lupa memperbaruinya saat menghapus mata kuliah. Data turunan cukup ditulis sebagai getter.
> - Salah path import (misalnya `import 'krs_controller.dart';` dari folder `views/`). Dari `views/` gunakan `'../controllers/krs_controller.dart'`, dari `main.dart` gunakan `'controllers/krs_controller.dart'`.

## 7. Lifecycle Controller & Bindings

Setiap `GetxController` punya siklus hidup: dibuat, dipakai, lalu dihapus dari memori. GetX memanggil method tertentu di setiap tahap, sehingga Anda tahu kapan harus memuat data dan kapan harus membersihkan sumber daya. Di section ini kita juga mengenal **Bindings**, tempat rapi untuk mendaftarkan banyak controller sekaligus.

### Tiga method lifecycle

| Method | Kapan dipanggil | Dipakai untuk |
|---|---|---|
| `onInit()` | Sekali, tepat setelah controller dibuat dan didaftarkan | Memuat data awal, memulai timer, membuat worker (section 9), berlangganan stream |
| `onReady()` | Sekali, satu frame setelah `onInit` (halaman sudah tampil) | Aksi yang butuh UI sudah siap, misalnya menampilkan dialog sambutan dari View |
| `onClose()` | Sekali, sebelum controller dihapus dari memori | Membatalkan `Timer`, `StreamSubscription`, worker, menutup `TextEditingController` milik controller |

Jika Anda pernah memakai `StatefulWidget`, polanya mirip: `onInit` setara `initState`, dan `onClose` setara `dispose`. Bedanya, lifecycle ini milik controller, bukan milik widget.

### Contoh kecil

```dart
import 'package:flutter/foundation.dart';
import 'package:get/get.dart';

class ContohController extends GetxController {
  ContohController() {
    debugPrint('1. constructor');
  }

  @override
  void onInit() {
    super.onInit(); // selalu panggil super lebih dulu
    debugPrint('2. onInit: muat data di sini');
  }

  @override
  void onReady() {
    super.onReady();
    debugPrint('3. onReady: halaman sudah tampil');
  }

  @override
  void onClose() {
```

```
    debugPrint('4. onClose: bersihkan sumber daya di sini');
    super.onClose();
  }
}
```

Kapan `onClose` dipanggil? Itu bergantung pada *cara* controller didaftarkan:

- Didaftarkan dengan `permanent: true` atau di `initialBinding` : hidup selama aplikasi berjalan, `onClose` praktis tidak pernah dipanggil.
- Didaftarkan lewat **binding sebuah rute** (atau `Get.put` di dalam halaman yang dibuka dengan `Get.to` / `Get.toNamed` ): GetX mengikat controller ke rute tersebut. Saat halaman ditutup ( `Get.back` atau tombol kembali), controller otomatis dihapus dan `onClose` dipanggil.
- Dihapus manual dengan `Get.delete<ContohController>()` .

**Bindings: satu tempat untuk mendaftarkan controller**

Di section 4 controller didaftarkan dengan `Get.put` di `main()` atau di View. Untuk aplikasi yang punya banyak controller, pendaftaran itu lebih rapi dikumpulkan dalam class turunan `Bindings` . Method `dependencies()` berisi semua `Get.put` / `Get.lazyPut` .

```
// 1. Binding global: dijalankan sekali saat aplikasi mulai
class AppBinding extends Bindings {
  @override
  void dependencies() {
    Get.put(TemaController(), permanent: true);
    Get.put(SesiController(), permanent: true);
    Get.lazyPut(() => FavoritController(), fenix: true);
  }
}

// 2. Binding per halaman: dijalankan saat rute dibuka
class BelajarBinding extends Bindings {
  @override
  void dependencies() {
    Get.lazyPut(() => BelajarController());
  }
}

// 3. Memasang binding
GetMaterialApp(
  initialBinding: AppBinding(),            // binding global
  initialRoute: '/',
  getPages: [
    GetPage(name: '/', page: () => const BerandaView()),
    GetPage(
      name: '/belajar',
      page: () => const BelajarView(),
      binding: BelajarBinding(),           // binding milik rute ini
    ),
  ],
);
```

```
// 4. Membuka rute bernama (boleh membawa arguments)
Get.toNamed('/belajar', arguments: 'Basis Data');
```

Untuk binding yang isinya hanya satu-dua baris, tidak perlu membuat class. Pakai `BindingsBuilder`:

```
GetPage(
  name: '/belajar',
  page: () => const BelajarView(),
  binding: BindingsBuilder(() {
    Get.lazyPut(() => BelajarController());
  }),
),
```

Jika Anda pernah melihat `MultiProvider` di package Provider, `AppBinding` inilah padanannya: semua controller global didaftarkan di satu tempat, tanpa widget yang bersarang.

**Contoh lengkap: tiga controller global + satu controller per halaman**

Aplikasi berikut punya `TemaController`, `SesiController`, dan `FavoritController` yang didaftarkan di `AppBinding`. Mengetuk nama mata kuliah membuka halaman Belajar yang punya `BelajarController` sendiri. Controller ini menjalankan `Timer` dan otomatis dihapus ketika halaman ditutup.

`lib/main.dart`

```
// lib/main.dart — salin SELURUH kode ini untuk menggantikan isi lib/main.dart
import 'dart:async';

import 'package:flutter/material.dart';
import 'package:get/get.dart';

// ================ CONTROLLER GLOBAL ================

// Controller 1: pengaturan tampilan
class TemaController extends GetxController {
  final _gelap = false.obs;
  bool get gelap => _gelap.value;

  void toggle() => _gelap.value = !_gelap.value;

  @override
  void onInit() {
    super.onInit();
    debugPrint('TemaController.onInit');
  }
}

// Controller 2: data pengguna yang sedang login
class SesiController extends GetxController {
  final _nama = RxnString(); // Rxn = observable yang boleh null
  String? get nama => _nama.value;
  bool get sudahLogin => _nama.value != null;
```

```dart
  void login(String nama) => _nama.value = nama;
  void logout() => _nama.value = null;

  @override
  void onInit() {
    super.onInit();
    debugPrint('SesiController.onInit');
  }
}

// Controller 3: daftar favorit
class FavoritController extends GetxController {
  final _items = <String>{}.obs; // RxSet
  int get jumlah => _items.length;
  bool isFavorit(String m) => _items.contains(m);

  void toggle(String m) {
    if (!_items.remove(m)) _items.add(m);
  }

  @override
  void onInit() {
    super.onInit();
    debugPrint('FavoritController.onInit');
  }
}

// ================= CONTROLLER PER HALAMAN =================

// Hidup hanya selama halaman Belajar terbuka
class BelajarController extends GetxController {
  final _detik = 0.obs;
  int get detik => _detik.value;

  Timer? _timer;

  @override
  void onInit() {
    super.onInit();
    debugPrint('BelajarController.onInit: timer dimulai');
    _timer = Timer.periodic(const Duration(seconds: 1), (_) => _detik.value++);
  }

  @override
  void onReady() {
    super.onReady();
    debugPrint('BelajarController.onReady: halaman sudah tampil');
  }

  @override
  void onClose() {
    _timer?.cancel(); // WAJIB: kalau lupa, timer tetap berjalan di latar
    debugPrint('BelajarController.onClose: timer dihentikan setelah $detik detik');
    super.onClose();
  }
}

// ================= BINDINGS =================
```

```dart
// Semua controller global didaftarkan di satu tempat
class AppBinding extends Bindings {
  @override
  void dependencies() {
    Get.put(TemaController(), permanent: true);
    Get.put(SesiController(), permanent: true);
    Get.put(FavoritController(), permanent: true);
  }
}

// Binding khusus halaman Belajar
class BelajarBinding extends Bindings {
  @override
  void dependencies() {
    Get.lazyPut(() => BelajarController());
  }
}

// ================= APP =================

void main() {
  runApp(const MyApp());
}

class MyApp extends StatelessWidget {
  const MyApp({super.key});

  @override
  Widget build(BuildContext context) {
    return GetMaterialApp(
      debugShowCheckedModeBanner: false,
      theme: ThemeData(colorSchemeSeed: Colors.indigo),
      darkTheme: ThemeData(colorSchemeSeed: Colors.indigo, brightness: Brightness.dark),
      initialBinding: AppBinding(), // dijalankan sekali saat aplikasi mulai
      initialRoute: '/',
      getPages: [
        GetPage(name: '/', page: () => const BerandaView()),
        GetPage(
          name: '/belajar',
          page: () => const BelajarView(),
          binding: BelajarBinding(), // controller dibuat saat rute dibuka
        ),
      ],
    );
  }
}

// ================= VIEW =================

class BerandaView extends StatelessWidget {
  const BerandaView({super.key});

  static const matkul = ['Pemrograman Mobile', 'Basis Data', 'Kecerdasan Buatan'];

  @override
  Widget build(BuildContext context) {
    final tema = Get.find<TemaController>();
    final sesi = Get.find<SesiController>();
    final favorit = Get.find<FavoritController>();
```

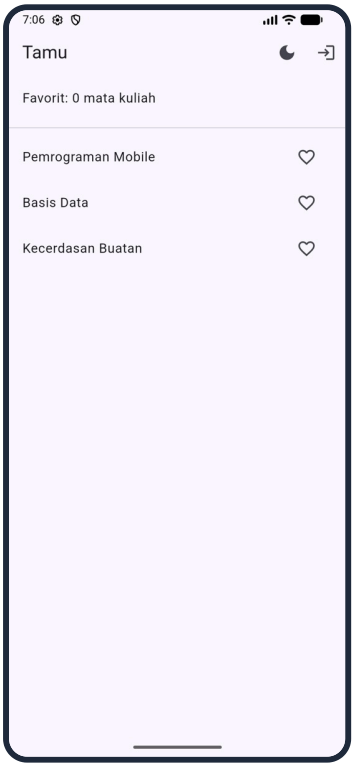
```
    return Scaffold(
      appBar: AppBar(
        title: Obx(() => Text(sesi.sudahLogin ? 'Halo, ${sesi.nama}' : 'Tamu')),
        actions: [
          IconButton(
            tooltip: 'Ganti tema',
            icon: const Icon(Icons.dark_mode),
            onPressed: () {
              tema.toggle(); // controller menyimpan state
              // View yang menerapkan state ke tampilan
              Get.changeThemeMode(tema.gelap ? ThemeMode.dark : ThemeMode.light);
            },
          ),
          Obx(
            () => IconButton(
              tooltip: sesi.sudahLogin ? 'Logout' : 'Login',
              icon: Icon(sesi.sudahLogin ? Icons.logout : Icons.login),
              onPressed: () => sesi.sudahLogin ? sesi.logout() : sesi.login('Andi'),
            ),
          ),
        ],
      ),
      body: ListView(
        children: [
          Obx(() => ListTile(title: Text('Favorit: ${favorit.jumlah} mata kuliah'))),
          const Divider(),
          for (final m in matkul)
            ListTile(
              title: Text(m),
              // ketuk baris untuk membuka halaman Belajar
              onTap: () => Get.toNamed('/belajar', arguments: m),
              trailing: Obx(
                () => IconButton(
                  tooltip: 'Favorit $m',
                  icon: Icon(
                    favorit.isFavorit(m) ? Icons.favorite : Icons.favorite_border,
                    color: favorit.isFavorit(m) ? Colors.red : null,
                  ),
                  onPressed: () => favorit.toggle(m),
                ),
              ),
            ),
        ],
      ),
    );
  }
}

// GetView<T> = StatelessWidget yang punya getter `controller` (= Get.find<T>())
class BelajarView extends GetView<BelajarController> {
  const BelajarView({super.key});

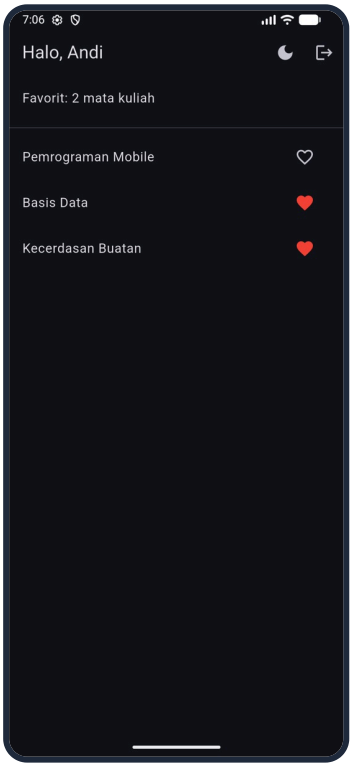
  @override
  Widget build(BuildContext context) {
    final matkul = Get.arguments as String? ?? '-';

    return Scaffold(
      appBar: AppBar(title: Text('Belajar $matkul')),
```

```
      body: Center(
        child: Column(
          mainAxisSize: MainAxisSize.min,
          children: [
            const Icon(Icons.timer, size: 64),
            const SizedBox(height: 12),
            Obx(
              () => Text(
                '${controller.detik} detik',
                style: Theme.of(context).textTheme.displaySmall,
              ),
            ),
            const SizedBox(height: 8),
            const Text('Tekan tombol kembali, lalu lihat log di konsol'),
          ],
        ),
      ),
    );
  }
}
```




Tiga controller global dari AppBinding




Setelah login, menambah favorit, dan ganti tema




Halaman Belajar: BelajarController dibuat oleh BelajarBinding

Log di konsol (*Debug Console* / terminal `flutter run`) setelah aplikasi dibuka, halaman Basis Data dibuka, lalu tombol kembali ditekan:

```
I/flutter: TemaController.onInit
I/flutter: SesiController.onInit
I/flutter: FavoritController.onInit
I/flutter: BelajarController.onInit: timer dimulai
I/flutter: BelajarController.onReady: halaman sudah tampil
I/flutter: BelajarController.onClose: timer dihentikan setelah 5 detik
```

**Penjelasan:**

- `initialBinding: AppBinding()` menjalankan `dependencies()` sekali saat aplikasi mulai, sehingga tiga controller global langsung tersedia untuk `Get.find` di View mana pun.
- Setiap controller kecil dan punya satu tanggung jawab. Hindari satu "GodController" berisi semua state.
- `BelajarBinding` dipasang di `GetPage`. Controller baru dibuat ketika rute `/belajar` dibuka, dan dihapus ketika rute ditutup. Buka halaman itu lagi: hitungan mulai dari 0 karena controllernya baru.
- `onClose` membatalkan `Timer`. Tanpa baris itu, timer tetap berdetak di latar dan terus mengubah observable milik controller yang sudah tidak dipakai (kebocoran memori).
- `BelajarView extends GetView<BelajarController>`: `GetView` hanyalah `StatelessWidget` yang menyediakan getter `controller` berisi `Get.find<BelajarController>()`.
- Tema: `TemaController` hanya menyimpan nilai `gelap`. Yang memanggil `Get.changeThemeMode` adalah View, karena itu urusan tampilan (prinsip MVC dari section 2).
- `Get.arguments` berisi nilai yang dikirim lewat `Get.toNamed('/belajar', arguments: m)`.
- Setiap `Obx` hanya membungkus bagian yang membaca observable: judul AppBar, ikon login, jumlah favorit, dan ikon hati per baris.

> ℹ️ **Get.put vs Get.lazyPut di binding**
>
> `Get.put` langsung membuat objek (dan menjalankan `onInit`) saat binding dijalankan. `Get.lazyPut` hanya mendaftarkan "resep"; objek baru dibuat saat pertama kali di-`Get.find`. Tambahkan `fenix: true` agar controller lazy bisa dibuat ulang setelah sempat dihapus.

⚠️ **Kesalahan umum**

- Lupa memanggil `super.onInit()` / `super.onClose()`. Selalu panggil `super`.
- Membuat `Timer`, `StreamSubscription`, atau worker di `onInit` tetapi tidak membatalkannya di `onClose`.
- Memasang controller khusus satu halaman di `AppBinding` dengan `permanent: true`. Controller itu tidak pernah dihapus dan state lamanya muncul lagi saat halaman dibuka ulang.
- Memakai `Get.toNamed('/belajar')` padahal rute itu tidak terdaftar di `getPages`, atau binding-nya lupa dipasang sehingga `Get.find` di View gagal dengan pesan *"BelajarController" not found*.

## 8. State Async: isLoading, error, data

Data dari internet atau database tidak datang seketika. Selama menunggu, View harus menampilkan **loading**; jika gagal, menampilkan **error** dengan tombol coba lagi; jika berhasil, menampilkan **data**. Pola tiga serangkai `isLoading` / `error` / `data` ini disimpan di controller dan akan dipakai ulang di Modul 09 (REST API) serta proyek CRUD Modul 13–14.

### Pola di sisi View

```dart
// Di dalam Obx. Urutan pengecekan: loading -> error -> kosong -> data
Obx(() {
  if (controller.isLoading) return const CircularProgressIndicator();
  if (controller.error != null) return Text(controller.error!);
  if (controller.data.isEmpty) return const Text('Belum ada data');
  return ListView(children: [for (final d in controller.data) Text(d.nama)]);
})
```

### Pola di sisi controller

1. Set `_isLoading.value = true` dan kosongkan `_error.value`. Karena keduanya observable, `Obx` langsung menampilkan loading tanpa perlu memanggil method pemberi tahu apa pun.
2. `try` : panggil service dan simpan hasilnya dengan `_data.assignAll(...)`.
3. `catch` : simpan pesan error.
4. `finally` : set `_isLoading.value = false`. Bagian `finally` selalu dijalankan, sehingga loading tidak "macet".
5. Panggil method pemuatan itu di `onInit()` agar data dimuat sekali saat controller dibuat (section 7).

### Contoh lengkap (multi-file)

```
lib/
├── main.dart
├── models/mahasiswa.dart
├── services/mahasiswa_service.dart
├── controllers/mahasiswa_controller.dart
├── bindings/app_binding.dart
└── views/mahasiswa_view.dart
```

**`lib/models/mahasiswa.dart`**

```dart
// lib/models/mahasiswa.dart
class Mahasiswa {
  final String nim;
  final String nama;
  final double ipk;

  const Mahasiswa({required this.nim, required this.nama, required this.ipk});
}
```

**lib/services/mahasiswa_service.dart**

```dart
// lib/services/mahasiswa_service.dart
// Service = satu-satunya tempat yang tahu cara MENGAMBIL data.
// Saat ini datanya palsu; di Modul 09 isi fungsi ini diganti dengan HTTP request.
import '../models/mahasiswa.dart';

class MahasiswaService {
  Future<List<Mahasiswa>> ambilSemua({bool simulasiGagal = false}) async {
    await Future.delayed(const Duration(seconds: 2)); // pura-pura menunggu server

    if (simulasiGagal) {
      throw Exception('Server tidak dapat dihubungi');
    }

    return const [
      Mahasiswa(nim: 'H1D023001', nama: 'Andi Pratama', ipk: 3.45),
      Mahasiswa(nim: 'H1D023002', nama: 'Bunga Lestari', ipk: 3.81),
      Mahasiswa(nim: 'H1D023003', nama: 'Citra Ayu', ipk: 3.12),
      Mahasiswa(nim: 'H1D023004', nama: 'Dimas Saputra', ipk: 3.67),
    ];
  }
}
```

**lib/controllers/mahasiswa_controller.dart**

```dart
// lib/controllers/mahasiswa_controller.dart
import 'package:get/get.dart';

import '../models/mahasiswa.dart';
import '../services/mahasiswa_service.dart';

class MahasiswaController extends GetxController {
  MahasiswaController(this._service); // service disuntikkan lewat constructor

  final MahasiswaService _service;

  // Tiga serangkai state async, semuanya observable
  final _isLoading = false.obs;
  final _error = RxnString(); // null = tidak ada error
  final _data = <Mahasiswa>[].obs;

  bool get isLoading => _isLoading.value;
  String? get error => _error.value;
  List<Mahasiswa> get data => List.unmodifiable(_data);

  @override
  void onInit() {
    super.onInit();
    muatData(); // pemuatan awal: sekali saat controller dibuat
  }

  Future<void> muatData({bool simulasiGagal = false}) async {
    if (_isLoading.value) return; // cegah request ganda

    _isLoading.value = true; // Obx langsung menampilkan loading
    _error.value = null; // hapus error lama
```

```
    try {
      final hasil = await _service.ambilSemua(simulasiGagal: simulasiGagal);
      _data.assignAll(hasil); // ganti seluruh isi list sekaligus
    } catch (e) {
      _error.value = 'Gagal memuat data: ${e.toString().replaceFirst('Exception: ', '')}';
    } finally {
      _isLoading.value = false; // SELALU dimatikan, berhasil maupun gagal
    }
  }
}
```

**lib/bindings/app_binding.dart**

```
// lib/bindings/app_binding.dart
import 'package:get/get.dart';

import '../controllers/mahasiswa_controller.dart';
import '../services/mahasiswa_service.dart';

class AppBinding extends Bindings {
  @override
  void dependencies() {
    Get.put(MahasiswaController(MahasiswaService()));
  }
}
```

**lib/views/mahasiswa_view.dart**

```
// lib/views/mahasiswa_view.dart
import 'package:flutter/material.dart';
import 'package:get/get.dart';

import '../controllers/mahasiswa_controller.dart';

class MahasiswaView extends GetView<MahasiswaController> {
  const MahasiswaView({super.key});

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(
        title: const Text('Data Mahasiswa'),
        actions: [
          IconButton(
            tooltip: 'Simulasi gagal',
            icon: const Icon(Icons.wifi_off),
            onPressed: () => controller.muatData(simulasiGagal: true),
          ),
        ],
      ),
      // Satu Obx untuk isi halaman: membaca isLoading, error, dan data
      body: Obx(() {
        // 1. Loading pertama kali (belum ada data sama sekali)
        if (controller.isLoading && controller.data.isEmpty) {
          return const Center(child: CircularProgressIndicator());
        }
```

```
        // 2. Error
        if (controller.error != null) {
          return _TampilanError(
            pesan: controller.error!,
            onCobaLagi: controller.muatData,
          );
        }

        // 3. Data kosong
        if (controller.data.isEmpty) {
          return const Center(child: Text('Belum ada data'));
        }

        // 4. Data tersedia (+ tarik ke bawah untuk memuat ulang)
        final data = controller.data;
        return RefreshIndicator(
          onRefresh: controller.muatData, // harus mengembalikan Future
          child: ListView.builder(
            itemCount: data.length,
            itemBuilder: (context, i) {
              final m = data[i];
              return ListTile(
                leading: CircleAvatar(child: Text(m.nama[0])),
                title: Text(m.nama),
                subtitle: Text(m.nim),
                trailing: Text('IPK ${m.ipk.toStringAsFixed(2)}'),
              );
            },
          ),
        );
      }),
    );
  }
}

class _TampilanError extends StatelessWidget {
  const _TampilanError({required this.pesan, required this.onCobaLagi});

  final String pesan;
  final VoidCallback onCobaLagi;

  @override
  Widget build(BuildContext context) {
    return Center(
      child: Padding(
        padding: const EdgeInsets.all(24),
        child: Column(
          mainAxisSize: MainAxisSize.min,
          children: [
            const Icon(Icons.error_outline, size: 64, color: Colors.red),
            const SizedBox(height: 12),
            Text(pesan, textAlign: TextAlign.center),
            const SizedBox(height: 12),
            FilledButton.icon(
              icon: const Icon(Icons.refresh),
              label: const Text('Coba Lagi'),
              onPressed: onCobaLagi,
            ),
          ],
```

```
        ),
      ),
    );
  }
}
```

**lib/main.dart**

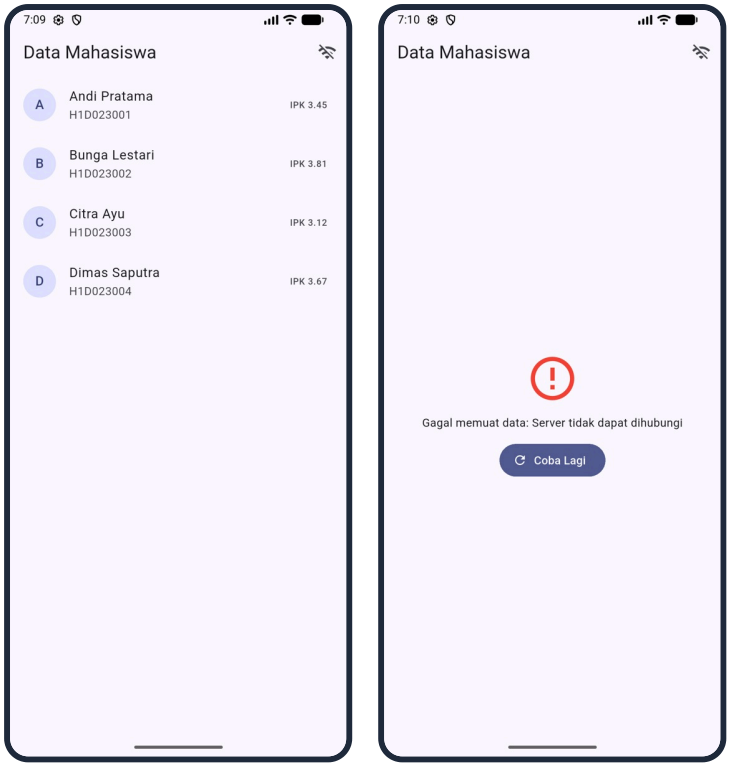
```dart
// lib/main.dart
import 'package:flutter/material.dart';
import 'package:get/get.dart';

import 'bindings/app_binding.dart';
import 'views/mahasiswa_view.dart';

void main() {
  runApp(const MyApp());
}

class MyApp extends StatelessWidget {
  const MyApp({super.key});

  @override
  Widget build(BuildContext context) {
    return GetMaterialApp(
      debugShowCheckedModeBanner: false,
      theme: ThemeData(colorSchemeSeed: Colors.indigo),
      initialBinding: AppBinding(),
      home: const MahasiswaView(),
    );
  }
}
```




State data (setelah loading 2 detik)

State error setelah ikon wifi ditekan, lengkap dengan tombol Coba Lagi

**Penjelasan:**

- **Model** (`Mahasiswa`) adalah Dart murni. **Service** hanya tahu cara mengambil data; ia tidak tahu apa pun soal GetX atau widget.
- **Controller** menerima service lewat constructor (*dependency injection*). Saat testing (Modul 15), service ini bisa diganti dengan versi palsu tanpa mengubah controller.
- `onInit()` memanggil `muatData()`. Tidak perlu `initState` atau `addPostFrameCallback` di View, karena controller sudah tahu kapan ia dibuat.
- `RxnString()` adalah observable yang boleh bernilai `null`. `null` berarti "tidak ada error".
- `_data.assignAll(hasil)` mengganti seluruh isi `RxList` dan memicu satu kali rebuild.
- `if (_isLoading.value) return;` mencegah request ganda saat tombol ditekan berkali-kali.
- View memakai satu `Obx` untuk isi halaman. Obx itu otomatis mendengarkan `isLoading`, `error`, dan `data` karena ketiganya dibaca di dalam builder.
- Kondisi loading ditulis `isLoading && data.isEmpty`: saat refresh (data lama masih ada), daftar tetap tampil dan yang berputar adalah indikator `RefreshIndicator`.
- `onRefresh: controller.muatData` bisa langsung dipakai karena `muatData` mengembalikan `Future`; indikator berhenti berputar saat Future selesai.
- Tombol ikon wifi mensimulasikan kegagalan sehingga tampilan error bisa dicoba tanpa mematikan internet. Tekan **Coba Lagi** untuk memuat ulang dengan normal.

💡 **Variasi: enum status**

Sebagian tim memakai `final status = Status.awal.obs;` dengan `enum Status { awal, loading, sukses, gagal }` sebagai pengganti beberapa boolean. Keduanya sah; yang penting View selalu menangani semua kemungkinan. GetX juga punya `StateMixin` dengan `obx(onLoading:, onError:)`, tetapi kursus ini memakai pola eksplisit di atas agar mudah dipindah ke state management lain.

⚠️ **Kesalahan umum**

- Tidak memakai `finally`: jika terjadi error, `isLoading` tetap `true` selamanya dan View terjebak di loading.
- Tidak mengosongkan `_error.value` saat mencoba lagi, sehingga pesan error lama tetap tampil walaupun request kedua berhasil.
- Memanggil `muatData()` di `build()` View. Setiap rebuild memicu request baru, dan karena `isLoading` berubah, Obx rebuild lagi: request tidak pernah berhenti. Tempatnya di `onInit`.
- Memanggil service langsung dari View. View hanya boleh bicara dengan controller.

## 9. Komunikasi Antar Controller & Workers

Kadang satu controller butuh data dari controller lain. Contoh: daftar catatan bergantung pada siapa yang sedang login. Di GetX, controller cukup mengambil controller lain dengan `Get.find`, lalu memasang **worker** agar otomatis bereaksi setiap kali observable di controller sumber berubah.

### Mengambil controller lain

```
class CatatanController extends GetxController {
  late final AuthController _auth;

  @override
  void onInit() {
    super.onInit();
    _auth = Get.find<AuthController>(); // controller yang SUDAH terdaftar
  }
}
```

Syaratnya: `AuthController` sudah didaftarkan sebelum `CatatanController.onInit` berjalan. Di binding, tulis `Get.put(AuthController())` lebih dulu, atau daftarkan dengan `Get.lazyPut` sehingga urutan tidak lagi penting.

### Worker: bereaksi terhadap perubahan observable

Worker adalah fungsi yang "mengawasi" sebuah observable dan menjalankan callback ketika nilainya berubah. Semuanya mengembalikan objek `Worker` yang harus dihentikan dengan `dispose()` di `onClose`.

| Worker | Callback dijalankan | Contoh pemakaian |
|---|---|---|
| `ever(rx, cb)` | Setiap kali nilai berubah | Pengguna berganti, muat ulang data miliknya |
| `once(rx, cb)` | Hanya pada perubahan pertama, lalu berhenti sendiri | Menampilkan tutorial saat pertama kali login |
| `debounce(rx, cb, time: ...)` | Setelah nilai *berhenti* berubah selama `time` (bawaan 800 ms) | Kotak pencarian: cari setelah pengguna selesai mengetik |
| `interval(rx, cb, time: ...)` | Paling banyak satu kali per `time` (bawaan 1 detik), perubahan di antaranya diabaikan | Membatasi tombol yang ditekan beruntun |

```
final _workers = <Worker>[];

@override
```

```
void onInit() {
  super.onInit();
  _workers.addAll([
    ever<String?>(_auth.nimRx, (nim) => debugPrint('login sebagai $nim')),
    once<String?>(_auth.nimRx, (_) => debugPrint('login pertama kali')),
    debounce<String>(_kataKunci, (k) => debugPrint('cari $k'),
        time: const Duration(milliseconds: 500)),
    interval<int>(_jumlahKlik, (n) => debugPrint('klik ke-$n'),
        time: const Duration(seconds: 1)),
  ]);
}

@override
void onClose() {
  for (final w in _workers) {
    w.dispose(); // hentikan semua worker
  }
  super.onClose();
}
```

### Contoh lengkap: catatan per pengguna + pencarian

`AuthController` menyimpan NIM yang sedang login. `CatatanController` memakai `ever` untuk mengganti daftar catatan setiap kali NIM berubah, dan `debounce` agar pencarian baru diterapkan setelah pengguna berhenti mengetik.

**lib/main.dart**

```
// lib/main.dart — salin SELURUH kode ini untuk menggantikan isi lib/main.dart
import 'package:flutter/material.dart';
import 'package:get/get.dart';

// ================= CONTROLLER SUMBER =================
class AuthController extends GetxController {
  final _nim = RxnString();

  String? get nim => _nim.value;

  // Observable-nya dibuka (read-only lewat getter) agar controller lain
  // bisa memasang worker. Ubah nilainya HANYA lewat method di bawah.
  RxnString get nimRx => _nim;

  void loginSebagai(String nim) => _nim.value = nim;
}

// ================= CONTROLLER YANG BERGANTUNG =================
// CatatanController BERGANTUNG pada AuthController: catatan milik siapa?
class CatatanController extends GetxController {
  late final AuthController _auth;
  final _workers = <Worker>[];

  final Map<String, List<String>> _catatanPerNim = {};
  final _catatan = <String>[].obs; // catatan milik pengguna aktif
  final _kataKunci = ''.obs; // isi kotak pencarian (berubah tiap ketikan)
  final _kataTerapan = ''.obs; // kata kunci yang SUDAH diterapkan (setelah debounce)

  String get kataTerapan => _kataTerapan.value;
```

```dart
  int get jumlah => _catatan.length;

  List<String> get catatanTampil {
    final k = _kataTerapan.value.toLowerCase();
    if (k.isEmpty) return List.unmodifiable(_catatan);
    return _catatan.where((c) => c.toLowerCase().contains(k)).toList();
  }

  @override
  void onInit() {
    super.onInit();
    _auth = Get.find<AuthController>(); // ambil controller lain

    // ever: jalankan callback SETIAP kali nim berubah
    _workers.add(ever<String?>(_auth.nimRx, _gantiPengguna));

    // debounce: jalankan callback 500 ms setelah pengguna BERHENTI mengetik
    _workers.add(debounce<String>(
      _kataKunci,
      (kata) {
        debugPrint('mencari: "$kata"');
        _kataTerapan.value = kata;
      },
      time: const Duration(milliseconds: 500),
    ));

    _gantiPengguna(_auth.nim); // nilai awal
  }

  void _gantiPengguna(String? nim) {
    debugPrint('ever: pengguna berganti ke $nim');
    _catatan.assignAll(nim == null ? const [] : _catatanPerNim[nim] ?? const []);
  }

  void ubahKataKunci(String kata) => _kataKunci.value = kata;

  void tambah(String teks) {
    final nim = _auth.nim;
    if (nim == null) return;
    _catatanPerNim.putIfAbsent(nim, () => []).add(teks);
    _catatan.add(teks);
  }

  @override
  void onClose() {
    for (final w in _workers) {
      w.dispose(); // hentikan semua worker
    }
    super.onClose();
  }
}

// ================ BINDING ================
class AppBinding extends Bindings {
  @override
  void dependencies() {
    // AuthController didaftarkan LEBIH DULU karena dipakai di onInit CatatanController
    Get.put(AuthController());
    Get.put(CatatanController());
```

```
  }
}

void main() {
  runApp(const MyApp());
}

class MyApp extends StatelessWidget {
  const MyApp({super.key});

  @override
  Widget build(BuildContext context) {
    return GetMaterialApp(
      debugShowCheckedModeBanner: false,
      theme: ThemeData(colorSchemeSeed: Colors.purple),
      initialBinding: AppBinding(),
      home: const CatatanView(),
    );
  }
}

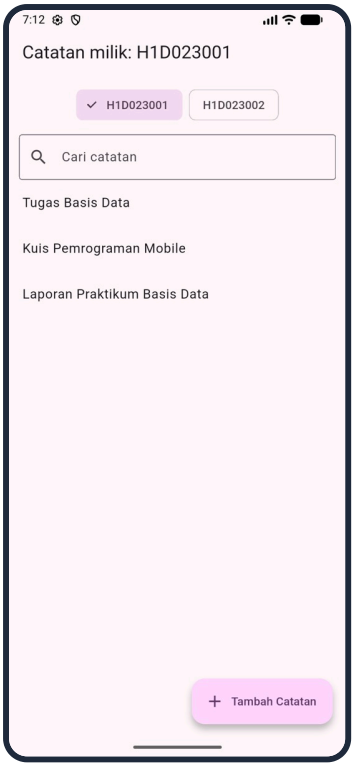
// ================= VIEW =================
class CatatanView extends StatelessWidget {
  const CatatanView({super.key});

  static const contoh = [
    'Tugas Basis Data',
    'Kuis Pemrograman Mobile',
    'Laporan Praktikum Basis Data',
    'Presentasi Kecerdasan Buatan',
  ];

  @override
  Widget build(BuildContext context) {
    final auth = Get.find<AuthController>();
    final catatan = Get.find<CatatanController>();

    return Scaffold(
      appBar: AppBar(title: Obx(() => Text('Catatan milik: ${auth.nim ?? '-'}'))),
      body: Column(
        children: [
          Padding(
            padding: const EdgeInsets.all(12),
            child: Obx(
              () => Wrap(
                spacing: 8,
                children: [
                  for (final n in ['H1D023001', 'H1D023002'])
                    ChoiceChip(
                      label: Text(n),
                      selected: auth.nim == n,
                      onSelected: (_) => auth.loginSebagai(n),
                    ),
                ],
              ),
            ),
          ),
          Padding(
            padding: const EdgeInsets.symmetric(horizontal: 12),
```

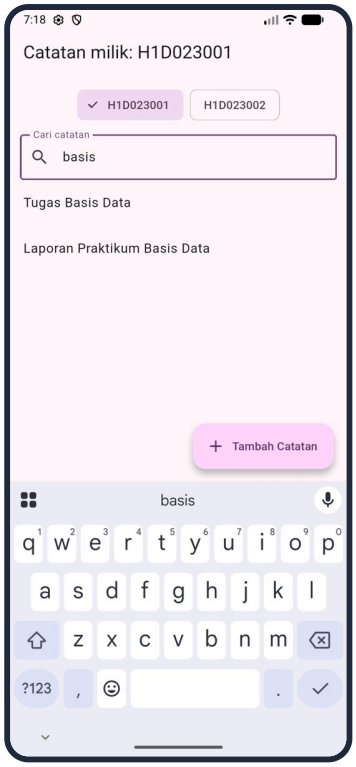
```
            child: TextField(
              decoration: const InputDecoration(
                labelText: 'Cari catatan',
                prefixIcon: Icon(Icons.search),
                border: OutlineInputBorder(),
              ),
              onChanged: catatan.ubahKataKunci,
            ),
          ),
          Expanded(
            child: Obx(() {
              final daftar = catatan.catatanTampil;
              if (daftar.isEmpty) {
                return const Center(child: Text('Belum ada catatan'));
              }
              return ListView(
                children: [for (final c in daftar) ListTile(title: Text(c))],
              );
            }),
          ),
        ],
      ),
      floatingActionButton: Obx(
        () => FloatingActionButton.extended(
          icon: const Icon(Icons.add),
          label: const Text('Tambah Catatan'),
          onPressed: auth.nim == null
              ? null
              : () => catatan.tambah(contoh[catatan.jumlah % contoh.length]),
        ),
      ),
    );
  }
}
```




Login sebagai H1D023001, tambah tiga catatan




Ganti ke H1D023002: worker ever mengganti daftar




Ketik "basis": debounce menerapkan pencarian

Log di konsol untuk skenario screenshot ketiga:

```
I/flutter: ever: pengguna berganti ke null
I/flutter: ever: pengguna berganti ke H1D023001
I/flutter: mencari: "basis"
```

Perhatikan: walaupun ada lima ketikan (`b`, `a`, `s`, `i`, `s`), callback debounce hanya berjalan **sekali**. Jika pencarian memanggil server (Modul 09), ini menghemat empat request yang sia-sia.

**Penjelasan:**

- `AppBinding` mendaftarkan `AuthController` lebih dulu karena `CatatanController.onInit` langsung memanggil `Get.find<AuthController>()`.
- `AuthController` membuka observable-nya lewat getter `nimRx` supaya bisa diawasi worker. Nilainya tetap hanya diubah lewat method `loginSebagai`.
- `ever<String?>(_auth.nimRx, _gantiPengguna)`: setiap kali NIM berubah, `_gantiPengguna` mengisi ulang `_catatan` dari map. Data per NIM tetap tersimpan, jadi kembali ke H1D023001 memunculkan tiga catatannya lagi.
- `ever` tidak dijalankan untuk nilai awal, hanya untuk perubahan. Karena itu `_gantiPengguna(_auth.nim)` dipanggil sekali secara manual di akhir `onInit` (baris log pertama, NIM masih `null`).
- Pencarian memakai dua observable: `_kataKunci` berubah setiap ketikan, sedangkan `_kataTerapan` baru berubah 500 ms setelah pengguna berhenti mengetik. `catatanTampil` hanya membaca `_kataTerapan`, jadi daftar tidak berkedip di setiap huruf.

- Semua worker disimpan di `_workers` dan dihentikan di `onClose`. Controller di contoh ini hidup sepanjang aplikasi, tetapi kebiasaan ini wajib untuk controller per halaman (section 7).
- `onChanged: catatan.ubahKataKunci` mengoper method sebagai callback (*tear-off*), tanpa membuat fungsi anonim.

> ℹ️ **Seberapa sering dipakai?**
>
> Untuk proyek kuliah, cara yang lebih sederhana juga sah: View memanggil `Get.find<CatatanController>().gantiPengguna(nim)` tepat setelah login berhasil. Worker `ever` lebih aman ketika status login bisa berubah dari banyak tempat (login, logout, sesi kedaluwarsa), karena controller tujuan pasti ikut bereaksi. Pola `ever` + auth ini muncul lagi di proyek Modul 14. Jika Anda pernah melihat `ProxyProvider` di package Provider, worker inilah padanannya di GetX.

> ⚠️ **Kesalahan umum**
>
> - Memasang worker di `build()` View. Setiap rebuild menambah worker baru, sehingga callback dipanggil berkali-kali. Worker selalu dibuat di `onInit` controller.
> - Lupa `dispose()` worker di `onClose`. Worker tetap mendengarkan observable milik controller lain yang masih hidup, sehingga callback berjalan pada controller yang sudah dihapus.
> - Urutan pendaftaran terbalik: `Get.put(CatatanController())` sebelum `Get.put(AuthController())` menghasilkan error *"AuthController" not found* di `onInit`.
> - Mengira `ever` langsung dijalankan untuk nilai awal. Panggil callback-nya sekali secara manual jika perlu.

## 10. Struktur Folder MVC & Auth Gate

Semakin besar aplikasi, semakin penting memisahkan kode berdasarkan tanggung jawabnya. Struktur MVC berikut sederhana, cukup untuk proyek kuliah, dan dipakai lagi di proyek CRUD + Auth (Modul 13–14).

```
lib/
├── main.dart              # runApp(GetMaterialApp(...)) + initialBinding
├── models/                # M: class data murni (User, Mahasiswa, Todo)
├── services/              # akses data: HTTP, database, Supabase, Hive
├── controllers/           # C: GetxController, state reaktif + logika bisnis
├── bindings/              # (opsional) class XxxBinding extends Bindings
├── views/                 # V: halaman penuh, file xxx_view.dart, class XxxView
└── widgets/               # komponen kecil yang dipakai ulang
```

| Folder | Boleh mengimpor | Tidak boleh |
|---|---|---|
| `models` | Dart murni | Flutter, GetX, service, controller |
| `services` | models, package http/database/Supabase | Widget, `BuildContext`, GetX, controller |
| `controllers` | models, services, `package:get/get.dart`, `foundation.dart` (debugPrint), controller lain lewat `Get.find` | `material.dart`, `BuildContext`, SnackBar/dialog, navigasi |
| `bindings` | controllers, services, GetX | Widget |
| `views` / `widgets` | models, controllers, GetX, widget lain | Memanggil service langsung |

Arah ketergantungan: **View → Controller → Service → Model**. Panah hanya menunjuk satu arah: controller tidak boleh tahu tentang View. Itulah sebabnya controller tidak menampilkan SnackBar atau bernavigasi sendiri. Controller mengembalikan hasil (`bool`, pesan error), lalu View yang memutuskan reaksinya.

> ℹ️ **"Tapi GetX bisa Get.snackbar dari controller..."**
>
> Benar. Karena `Get.snackbar`, `Get.to`, dan `Get.back` tidak butuh `BuildContext`, GetX sebenarnya mengizinkan semua itu dipanggil dari controller. Kursus ini **sengaja tidak melakukannya**. Controller yang bebas UI bisa diuji dengan unit test biasa tanpa widget (Modul 15), dan logikanya bisa dipakai ulang oleh View yang berbeda.

### Contoh lengkap: Auth gate dengan GetX

Auth gate adalah widget yang memilih menampilkan `LoginView` atau `HomeView` berdasarkan status sesi di `AuthController`. Login dan logout tidak memerlukan `Get.to` atau `Get.off` sama sekali.

```
lib/
├── main.dart
├── models/user.dart
├── services/auth_service.dart
├── controllers/auth_controller.dart
├── bindings/app_binding.dart
├── views/
|   ├── login_view.dart
|   └── home_view.dart
└── widgets/auth_gate.dart
```

**`lib/models/user.dart`**

```dart
// lib/models/user.dart
class User {
  final String email;
  final String nama;

  const User({required this.email, required this.nama});
}
```

**`lib/services/auth_service.dart`**

```dart
// lib/services/auth_service.dart
// Service palsu. Di Modul 12 bagian ini diganti dengan Supabase Auth.
import '../models/user.dart';

class AuthService {
  Future<User> login(String email, String password) async {
    await Future.delayed(const Duration(seconds: 1));
    if (password != '123456') {
      throw Exception('Email atau password salah');
    }
    return User(email: email, nama: email.split('@').first);
  }

  Future<void> logout() async {
```

```
    await Future.delayed(const Duration(milliseconds: 300));
  }
}
```

**lib/controllers/auth_controller.dart**

```dart
// lib/controllers/auth_controller.dart
// Controller TIDAK mengimpor material.dart dan tidak memegang BuildContext.
import 'package:get/get.dart';

import '../models/user.dart';
import '../services/auth_service.dart';

class AuthController extends GetxController {
  AuthController(this._service);

  final AuthService _service;

  final _user = Rxn<User>(); // null = belum login
  final _isLoading = false.obs;
  final _error = RxnString();

  User? get user => _user.value;
  bool get isLoggedIn => _user.value != null;
  bool get isLoading => _isLoading.value;
  String? get error => _error.value;

  /// Mengembalikan true jika berhasil. View yang memutuskan cara
  /// menampilkan kegagalan (SnackBar, dialog, dll).
  Future<bool> login(String email, String password) async {
    _isLoading.value = true;
    _error.value = null;
    try {
      _user.value = await _service.login(email, password);
      return true;
    } catch (e) {
      _error.value = e.toString().replaceFirst('Exception: ', '');
      return false;
    } finally {
      _isLoading.value = false;
    }
  }

  Future<void> logout() async {
    await _service.logout();
    _user.value = null;
  }
}
```

**lib/bindings/app_binding.dart**

```dart
// lib/bindings/app_binding.dart
import 'package:get/get.dart';

import '../controllers/auth_controller.dart';
import '../services/auth_service.dart';
```

```dart
class AppBinding extends Bindings {
  @override
  void dependencies() {
    Get.put(AuthController(AuthService()), permanent: true);
    // Controller global lain (KrsController, TodoController, ...) ditambahkan di sini
  }
}
```

**lib/widgets/auth_gate.dart**

```dart
// lib/widgets/auth_gate.dart
import 'package:flutter/material.dart';
import 'package:get/get.dart';

import '../controllers/auth_controller.dart';
import '../views/home_view.dart';
import '../views/login_view.dart';

// Auth gate: memilih halaman berdasarkan status sesi.
// Tidak ada Get.to / Get.off sama sekali untuk login/logout!
class AuthGate extends GetView<AuthController> {
  const AuthGate({super.key});

  @override
  Widget build(BuildContext context) {
    // Obx hanya membaca isLoggedIn, jadi hanya rebuild saat status login berubah
    return Obx(
      () => controller.isLoggedIn ? const HomeView() : const LoginView(),
    );
  }
}
```

**lib/views/login_view.dart**

```dart
// lib/views/login_view.dart
import 'package:flutter/material.dart';
import 'package:get/get.dart';

import '../controllers/auth_controller.dart';

// StatefulWidget karena TextEditingController adalah state LOKAL form ini
class LoginView extends StatefulWidget {
  const LoginView({super.key});

  @override
  State<LoginView> createState() => _LoginViewState();
}

class _LoginViewState extends State<LoginView> {
  final _auth = Get.find<AuthController>();
  final _emailController = TextEditingController(text: 'andi@mhs.unsoed.ac.id');
  final _passwordController = TextEditingController();

  @override
  void dispose() {
    _emailController.dispose();
    _passwordController.dispose();
```

```
    super.dispose();
  }

  Future<void> _login() async {
    FocusScope.of(context).unfocus(); // tutup keyboard
    final berhasil = await _auth.login(
      _emailController.text.trim(),
      _passwordController.text,
    );
    // Jika berhasil, AuthGate otomatis menampilkan HomeView.
    if (!berhasil && mounted) {
      ScaffoldMessenger.of(context).showSnackBar(
        SnackBar(content: Text(_auth.error ?? 'Login gagal')),
      );
    }
  }

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(title: const Text('Login')),
      body: Padding(
        padding: const EdgeInsets.all(24),
        child: Column(
          children: [
            TextField(
              controller: _emailController,
              decoration: const InputDecoration(labelText: 'Email'),
            ),
            TextField(
              controller: _passwordController,
              obscureText: true,
              decoration: const InputDecoration(
                labelText: 'Password',
                helperText: 'Petunjuk: 123456',
              ),
            ),
            const SizedBox(height: 24),
            SizedBox(
              width: double.infinity,
              // Obx kecil: hanya tombol yang rebuild saat isLoading berubah
              child: Obx(
                () => FilledButton(
                  onPressed: _auth.isLoading ? null : _login,
                  child: _auth.isLoading
                      ? const SizedBox(
                          width: 20,
                          height: 20,
                          child: CircularProgressIndicator(strokeWidth: 2),
                        )
                      : const Text('Masuk'),
                ),
              ),
            ),
          ],
        ),
      ),
    );
```

```
  }
}
```

**lib/views/home_view.dart**

```dart
// lib/views/home_view.dart
import 'package:flutter/material.dart';
import 'package:get/get.dart';

import '../controllers/auth_controller.dart';

class HomeView extends GetView<AuthController> {
  const HomeView({super.key});

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(
        title: const Text('Beranda'),
        actions: [
          IconButton(
            tooltip: 'Logout',
            icon: const Icon(Icons.logout),
            // Setelah logout, AuthGate otomatis kembali ke LoginView
            onPressed: controller.logout,
          ),
        ],
      ),
      body: Center(
        child: Obx(
          () => Text(
            'Selamat datang, ${controller.user?.nama}!',
            style: Theme.of(context).textTheme.headlineSmall,
          ),
        ),
      ),
    );
  }
}
```

**lib/main.dart**

```dart
// lib/main.dart
import 'package:flutter/material.dart';
import 'package:get/get.dart';

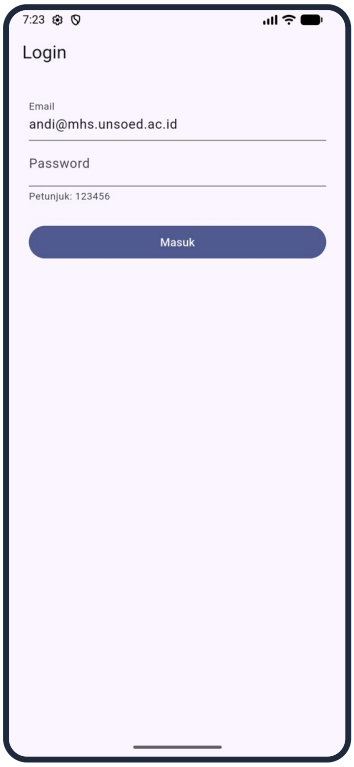
import 'bindings/app_binding.dart';
import 'widgets/auth_gate.dart';

void main() {
  runApp(const MyApp());
}

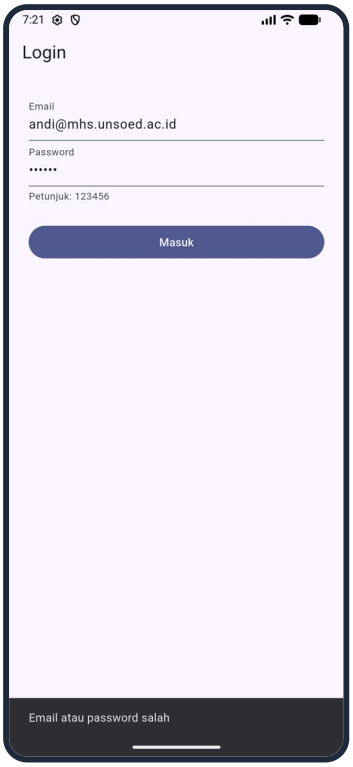
class MyApp extends StatelessWidget {
  const MyApp({super.key});

  @override
```

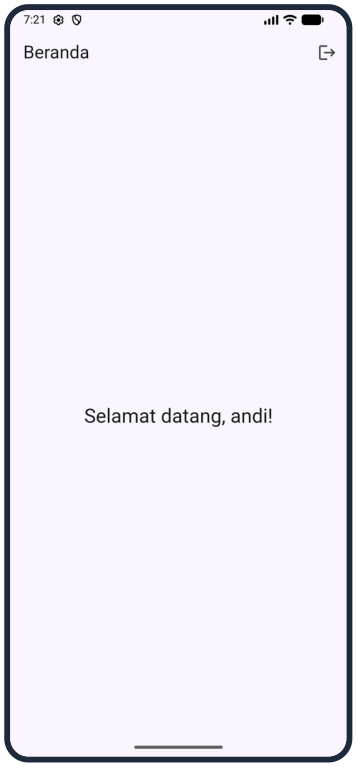
```
  Widget build(BuildContext context) {
    return GetMaterialApp(
      debugShowCheckedModeBanner: false,
      theme: ThemeData(colorSchemeSeed: Colors.indigo),
      initialBinding: AppBinding(),
      home: const AuthGate(),
    );
  }
}
```




AuthGate menampilkan LoginView




Password salah: View menampilkan pesan error dari controller




Login berhasil: AuthGate menampilkan HomeView

**Penjelasan:**

- `AuthController` menyimpan user dalam `Rxn<User>()`, observable yang boleh `null`. `null` berarti belum login.
- `AuthGate` membungkus pilihan halaman dengan `Obx` yang hanya membaca `isLoggedIn`. Begitu `_user.value` berubah, gate langsung berganti halaman. Saat `isLoading` berubah, gate tidak ikut rebuild.
- `login()` mengembalikan `bool`. `LoginView` yang memutuskan cara menampilkan kegagalan (SnackBar, dengan cek `mounted` setelah `await`), sedangkan controller hanya menyimpan pesannya di `error`.
- `LoginView` tetap `StatefulWidget` karena `TextEditingController` adalah state lokal milik form (ephemeral state, section 1). State global (siapa yang login) ada di controller.
- Tombol Masuk dibungkus `Obx` kecil: dinonaktifkan dan diganti spinner selama `isLoading`, mencegah login ganda.

- Logout cukup memanggil `controller.logout`. Begitu user menjadi `null`, AuthGate menampilkan LoginView lagi.
- `AppBinding` mendaftarkan `AuthController` dengan `permanent: true` karena sesi login dibutuhkan selama aplikasi hidup.
- `auth_controller.dart` hanya mengimpor `package:get/get.dart`, model, dan service. Coba cek: tidak ada `material.dart`.

> 💡 **Dipakai lagi di Modul 13–14**
>
> Proyek CRUD + Auth nanti memakai nama yang sama: `lib/controllers/auth_controller.dart` (`AuthController`), `AuthGate`, `LoginView`, `RegisterView`, `HomeView`, ditambah `NotesController`. Bedanya, `AuthService` palsu di atas diganti Supabase Auth, dan status login didengarkan dari stream (`StreamSubscription` dibuat di `onInit`, dibatalkan di `onClose`).

> ⚠️ **Kesalahan umum**
>
> - Mengimpor `material.dart` di controller untuk memanggil `ScaffoldMessenger`, atau memanggil `Get.snackbar` / `Get.to` dari controller. Kembalikan hasil (`bool`/pesan) dan biarkan View yang bereaksi.
> - Mencampur AuthGate dengan `Get.off(() => const HomeView())` manual setelah login. Pilih satu cara; jika memakai AuthGate, biarkan ia yang menentukan halaman.
> - Memanggil `AuthService` langsung dari `LoginView`. View hanya bicara dengan controller.
> - Menaruh semua class di `main.dart` sampai ratusan baris. Pecah sesuai struktur di atas sejak awal.

## 11. Kesalahan Umum & Cara Memperbaikinya

Bagian ini merangkum kesalahan GetX yang paling sering muncul di praktikum, lengkap dengan pesan error asli (diambil dari konsol `flutter run` dan `flutter analyze`) dan cara memperbaikinya.

### 1. Obx tanpa observable

```
class CounterController extends GetxController {
  int nilai = 0; // ❌ BUKAN observable
}

// di View
Obx(() => Text('Nilai: ${c.nilai}')) // ❌ tidak ada .obs yang dibaca
```

Begitu halaman dibuka, layar merah muncul dan konsol menampilkan:

```
══╡ EXCEPTION CAUGHT BY WIDGETS LIBRARY ╞═══════════════════
The following message was thrown building Obx(has builder, dirty, state: ObxState#33f31):
      [Get] the improper use of a GetX has been detected.
      You should only use GetX or Obx for the specific widget that will be updated.
      If you are seeing this error, you probably did not insert any observable variables into
GetX/Obx
      or insert them outside the scope that GetX considers suitable for an update
      (example: GetX => HeavyWidget => variableObservable).
      If you need to update a parent widget and a child widget, wrap each one in an Obx/GetX.
```

**Perbaikan:** jadikan field-nya observable ( `final nilai = 0.obs;` ), atau jika memang tidak perlu reaktif, buang `Obx` -nya. Error yang sama muncul bila observable dibaca di *luar* builder Obx, misalnya disimpan ke variabel lokal sebelum `Obx(() => ...)` , atau dibaca di dalam widget lain yang punya method `build` sendiri.

```
// ❌ dibaca di luar Obx: Obx tidak melihat observable apa pun
final teks = c.nama.value;
return Obx(() => Text(teks));

// ✅ dibaca di DALAM builder Obx
return Obx(() => Text(c.nama.value));
```

### 2. Lupa `.value` atau tertukar antara Rx dan nilainya

Sebagian besar kasus ini ditangkap `flutter analyze` sebelum aplikasi dijalankan:

```
final isLoading = false.obs;   // RxBool
final nama = 'Andi'.obs;       // RxString
final count = 0.obs;           // RxInt

if (c.isLoading) {}            // ❌ error: Conditions must have a static type of 'bool'.
String n = c.nama;             // ❌ error: A value of type 'RxString' can't be assigned
                               //    to a variable of type 'String'.
Text(c.nama);                  // ❌ error: The argument type 'RxString' can't be assigned
```

```
                                //    to the parameter type 'String'.
c.count = 3;                    // ❌ error: 'count' can't be used as a setter because it's final.
if (5 == c.count) {}            // ⚠️ info: unrelated_type_equality_checks, dan hasilnya SELALU false

// ✅ BENAR
if (c.isLoading.value) {}
String n = c.nama.value;
Text(c.nama.value);
c.count.value = 3;
if (c.count.value == 5) {}
```

Kasus yang **tidak** tertangkap analyzer dan paling menjebak: field Rx dideklarasikan dengan `var`, lalu diganti dengan objek Rx baru.

```
var skor = 0.obs;

void reset() {
  skor = 0.obs; // ❌ Obx masih mendengarkan objek Rx LAMA, layar tidak berubah
}

// ✅ selalu deklarasikan Rx dengan final, ubah isinya lewat .value
final skor = 0.obs;
void reset() => skor.value = 0;
```

> 💡 **Getter menyelamatkan Anda**
>
> Pola dari section 6 (field Rx privat + getter yang mengembalikan `.value`, misalnya `bool get isLoading => _isLoading.value;`) menghilangkan hampir semua kesalahan ini. View cukup menulis `controller.isLoading` tanpa memikirkan `.value`.

### 3. Get.find sebelum controller didaftarkan

```
void main() {
  runApp(const GetMaterialApp(home: HalamanSalah())); // ❌ tidak ada Get.put
}

// di HalamanSalah.build
final c = Get.find<KrsController>();
```

```
══╡ EXCEPTION CAUGHT BY WIDGETS LIBRARY ╞═══════════════════
The following message was thrown building HalamanSalah(dirty):
"KrsController" not found. You need to call "Get.put(KrsController())" or
"Get.lazyPut(()=>KrsController())"
```

**Perbaikan:** daftarkan controller lebih dulu: `Get.put` di `main()`, `initialBinding`, atau `binding` di `GetPage` / `Get.to` (section 4 dan 7). Periksa juga: apakah controller per halaman sudah terhapus karena halamannya ditutup, sementara halaman lain masih memanggil `Get.find`?

### 4. Mengubah field objek di dalam RxList

```
class Tugas {
  Tugas(this.judul);
  final String judul;
  bool selesai = false; // field mutable
}

final tugas = <Tugas>[Tugas('Laporan'), Tugas('Kuis')].obs;

void toggle(int i) {
  tugas[i].selesai = !tugas[i].selesai; // ❌ checkbox TIDAK berubah di layar
}
```

Tidak ada pesan error sama sekali, tetapi checkbox tidak tercentang. `RxList` hanya tahu ketika *isi list* berubah (tambah, hapus, ganti elemen). Mengubah field di dalam objek yang sama tidak terdeteksi. Dua perbaikan:

```
// ✅ Cara 1 (disarankan): model immutable + copyWith, lalu GANTI elemennya
_items[i] = _items[i].copyWith(selesai: !_items[i].selesai);

// ✅ Cara 2: tetap mutable, lalu beri tahu Obx secara manual
tugas[i].selesai = !tugas[i].selesai;
tugas.refresh();
```

Aturan yang sama berlaku untuk `Rx<ObjekBiasa>`: setelah mengubah field-nya, panggil `.refresh()` atau ganti seluruh objek lewat `.value`.

### 5. Get.put dan memuat data di dalam build

```
@override
Widget build(BuildContext context) {
  final c = Get.put(PengumumanController()); // ⚠️ objek baru dibuat setiap build
  return Scaffold(
    body: Obx(() {
      c.muat(); // ❌ dipanggil setiap Obx rebuild
      if (c.isLoading) return const Center(child: CircularProgressIndicator());
      return ListView(children: [for (final p in c.items) Text(p)]);
    }),
  );
}
```

Konsol menunjukkan request yang tidak pernah berhenti: `muat()` mengubah `isLoading`, Obx rebuild, rebuild memanggil `muat()` lagi, dan seterusnya.

```
I/flutter: constructor PengumumanController
I/flutter: muat() ke-1
I/flutter: muat() ke-2
I/flutter: muat() ke-3
I/flutter: muat() ke-4
...
```

**Perbaikan:** panggil `muat()` di `onInit()` controller (section 8). Untuk `Get.put` di `build`: aplikasinya tetap jalan karena `Get.put` tidak menimpa instance yang sudah terdaftar, tetapi argumen `PengumumanController()` tetap dibuat lalu dibuang setiap kali widget itu rebuild. Daftarkan controller di binding, lalu ambil dengan `Get.find` atau `GetView`.

### 6. Lupa membersihkan di onClose

```
class JamController extends GetxController {
  final detik = 0.obs;
  StreamSubscription<int>? _sub;

  @override
  void onInit() {
    super.onInit();
    _sub = Stream.periodic(const Duration(seconds: 1), (i) => i).listen((i) {
      detik.value = i;
      debugPrint('tick $i');
    });
  }

  @override
  void onClose() {
    debugPrint('onClose dipanggil');
    // ❌ lupa: _sub?.cancel();
    super.onClose();
  }
}
```

Halaman Jam dibuka, lalu ditutup setelah sekitar 3 detik. Controller sudah dihapus, tetapi stream tetap berjalan:

```
I/flutter: tick 0
I/flutter: tick 1
I/flutter: tick 2
I/flutter: tick 3
I/flutter: onClose dipanggil
I/flutter: tick 4      <- halaman sudah ditutup, tetapi masih berdetak
I/flutter: tick 5
I/flutter: tick 6
```

**Perbaikan:** batalkan semua yang Anda mulai di `onInit`: `_sub?.cancel()`, `_timer?.cancel()`, `worker.dispose()` (section 9), dan `dispose()` untuk `TextEditingController` yang dimiliki controller.

### 7. Controller memanggil UI

```
// ❌ SALAH: controller tahu tentang SnackBar dan navigasi
Future<void> login(String email, String password) async {
  try {
    _user.value = await _service.login(email, password);
    Get.offAll(() => const HomeView());
  } catch (e) {
    Get.snackbar('Gagal', e.toString());
```

```
  }
}

// ✅ BENAR: controller mengembalikan hasil, View yang bereaksi
Future<bool> login(String email, String password) async {
  try {
    _user.value = await _service.login(email, password);
    return true;
  } catch (e) {
    _error.value = e.toString().replaceFirst('Exception: ', '');
    return false;
  }
}
```

Versi salah tetap berjalan (GetX mengizinkannya), tetapi melanggar aturan MVC kursus ini: controller jadi bergantung pada View dan tidak bisa di-unit-test tanpa widget. Lihat contoh `AuthController` + `LoginView` di section 10.

### 8. Memakai MaterialApp padahal memanggil Get.to / Get.snackbar

```
runApp(const MaterialApp(home: HalamanSalah())); // ❌ bukan GetMaterialApp

// di dalam tombol
onPressed: () => Get.to(() => const DetailView()),
```

```
══╡ EXCEPTION CAUGHT BY GESTURE ╞═══════════════════════════
The following message was thrown while handling a gesture:
You are trying to use contextless navigation without
      a GetMaterialApp or Get.key.
      If you are testing your app, you can use:
      [Get.testMode = true], or if you are running your app on
      a physical device or emulator, you must exchange your [MaterialApp]
      for a [GetMaterialApp].
```

`Get.snackbar` di bawah `MaterialApp` bahkan memberi pesan yang lebih membingungkan:

```
E/flutter: Unhandled Exception: Null check operator used on a null value
E/flutter: #0      SnackbarController._configureOverlay
(package:get/get_navigation/src/snackbar/snackbar_controller.dart:94:18)
```

**Perbaikan:** ganti `MaterialApp` dengan `GetMaterialApp`. Semua parameter lama (`theme`, `home`, `debugShowCheckedModeBanner`, ...) tetap sama. `Obx`, `Get.put`, dan `Get.find` sebenarnya bekerja dengan `MaterialApp` biasa; yang butuh `GetMaterialApp` adalah fitur rute, binding, snackbar, dan dialog GetX.

### Contoh lengkap: semua perbaikan dalam satu aplikasi

Aplikasi Pengumuman berikut menerapkan perbaikan di atas: `GetMaterialApp`, controller per halaman lewat binding (dihapus otomatis), data dimuat di `onInit`, elemen `RxList` diganti dengan `copyWith`, dan halaman detail memakai controller yang sama lewat `GetView`.

**lib/main.dart**

```dart
// lib/main.dart — salin SELURUH kode ini untuk menggantikan isi lib/main.dart
// Contoh BENAR: controller per halaman lewat binding, data dimuat di onInit,
// elemen RxList diganti (bukan diubah isinya), halaman detail memakai Get.find.
import 'package:flutter/material.dart';
import 'package:get/get.dart';

// ---------- MODEL (immutable) ----------
class Pengumuman {
  final String id;
  final String judul;
  final bool dibaca;

  const Pengumuman({required this.id, required this.judul, this.dibaca = false});

  Pengumuman copyWith({bool? dibaca}) =>
      Pengumuman(id: id, judul: judul, dibaca: dibaca ?? this.dibaca);
}

// ---------- CONTROLLER ----------
class PengumumanController extends GetxController {
  final _isLoading = false.obs;
  final _items = <Pengumuman>[].obs;

  bool get isLoading => _isLoading.value;
  List<Pengumuman> get items => List.unmodifiable(_items);

  Pengumuman? cari(String id) => _items.firstWhereOrNull((p) => p.id == id);

  @override
  void onInit() {
    super.onInit();
    muat(); // ✅ sekali, bukan di build
  }

  Future<void> muat() async {
    _isLoading.value = true;
    try {
      await Future.delayed(const Duration(seconds: 1));
      _items.assignAll(const [
        Pengumuman(id: '1', judul: 'Libur Dies Natalis'),
        Pengumuman(id: '2', judul: 'Jadwal UTS'),
        Pengumuman(id: '3', judul: 'Beasiswa PPA'),
      ]);
    } finally {
      _isLoading.value = false;
    }
  }

  void tandaiDibaca(String id) {
    final i = _items.indexWhere((p) => p.id == id);
    if (i == -1) return;
    // ✅ GANTI elemennya, RxList langsung memberi tahu Obx
    _items[i] = _items[i].copyWith(dibaca: true);
  }

  @override
  void onClose() {
```

```
    debugPrint('PengumumanController dihapus');
    super.onClose();
  }
}

// ---------- BINDING ----------
class PengumumanBinding extends Bindings {
  @override
  void dependencies() {
    Get.lazyPut(() => PengumumanController());
  }
}

// ---------- APP ----------
void main() {
  runApp(const MyApp());
}

class MyApp extends StatelessWidget {
  const MyApp({super.key});

  @override
  Widget build(BuildContext context) {
    // ✅ GetMaterialApp, bukan MaterialApp, karena memakai Get.to
    return GetMaterialApp(
      debugShowCheckedModeBanner: false,
      theme: ThemeData(colorSchemeSeed: Colors.blueGrey),
      home: const MenuView(),
    );
  }
}

class MenuView extends StatelessWidget {
  const MenuView({super.key});

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(title: const Text('Menu')),
      body: Center(
        child: FilledButton(
          // ✅ controller dibuat oleh binding saat halaman dibuka,
          //    dan dihapus otomatis saat halaman ditutup
          onPressed: () => Get.to(
            () => const PengumumanView(),
            binding: PengumumanBinding(),
          ),
          child: const Text('Buka Pengumuman'),
        ),
      ),
    );
  }
}

class PengumumanView extends GetView<PengumumanController> {
  const PengumumanView({super.key});

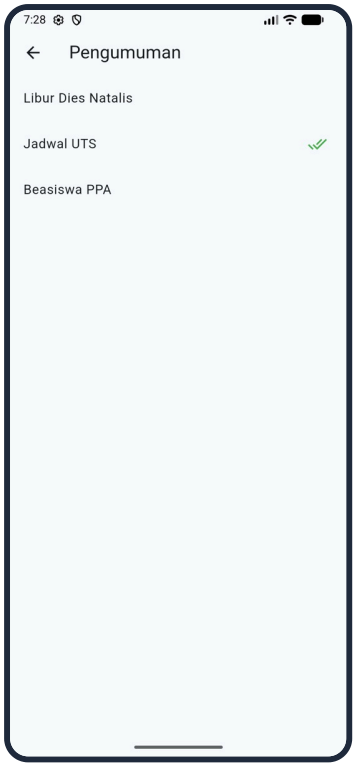
  @override
  Widget build(BuildContext context) {
```

```
    return Scaffold(
      appBar: AppBar(title: const Text('Pengumuman')),
      body: Obx(() {
        if (controller.isLoading) {
          return const Center(child: CircularProgressIndicator());
        }
        return ListView(
          children: [
            for (final p in controller.items)
              ListTile(
                title: Text(p.judul),
                trailing: p.dibaca
                    ? const Icon(Icons.done_all, color: Colors.green)
                    : null,
                // ✅ cukup kirim id; detail mengambil controller yang SAMA
                onTap: () => Get.to(() => DetailPengumumanView(id: p.id)),
              ),
          ],
        );
      }),
    );
  }
}

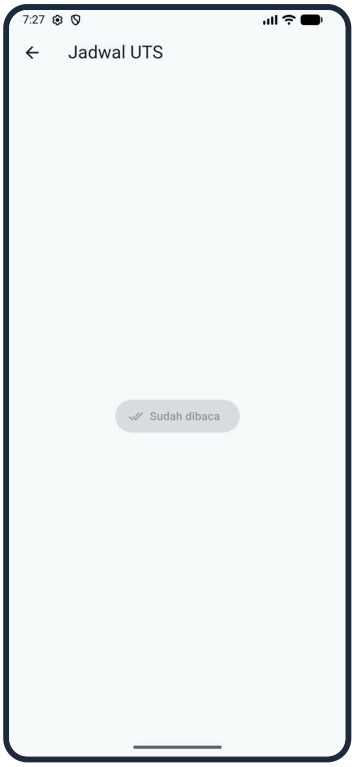
class DetailPengumumanView extends GetView<PengumumanController> {
  const DetailPengumumanView({super.key, required this.id});

  final String id;

  @override
  Widget build(BuildContext context) {
    return Obx(() {
      final p = controller.cari(id);
      if (p == null) return const Scaffold(body: Center(child: Text('Tidak ditemukan')));
      return Scaffold(
        appBar: AppBar(title: Text(p.judul)),
        body: Center(
          child: FilledButton.icon(
            icon: Icon(p.dibaca ? Icons.done_all : Icons.mark_email_read),
            label: Text(p.dibaca ? 'Sudah dibaca' : 'Tandai sudah dibaca'),
            onPressed: p.dibaca ? null : () => controller.tandaiDibaca(id),
          ),
        ),
      );
    });
  }
}
```




Kembali ke daftar: tanda dibaca langsung muncul




Halaman detail memakai controller yang sama (Get.find)

**Penjelasan:**

- `PengumumanBinding` dipasang di `Get.to(..., binding: ...)`. Controller dibuat saat halaman Pengumuman dibuka; saat halaman ditutup, konsol mencetak `PengumumanController dihapus`.
- `muat()` dipanggil sekali di `onInit`, dengan `finally` agar loading pasti berhenti.
- `Pengumuman` immutable. `tandaiDibaca` mengganti elemen list dengan salinan baru, sehingga semua `Obx` yang membaca `items` rebuild.
- `DetailPengumumanView` hanya menerima `id`, lalu mencari datanya di controller yang sama. Tidak ada objek yang dioper lewat constructor dan tidak ada yang perlu dikembalikan lewat `Get.back(result: ...)`.
- `firstWhereOrNull` berasal dari extension GetX (juga ada di package `collection`): mengembalikan `null` jika tidak ketemu, alih-alih melempar error.

### Ringkasan

| Gejala / pesan error | Penyebab | Perbaikan |
|---|---|---|
| *[Get] the improper use of a GetX has been detected* | Obx tidak membaca observable apa pun di dalam builder-nya | Pakai `.obs` dan baca di dalam builder, atau hapus Obx |
| *Conditions must have a static type of 'bool', A value of type 'RxString' can't be assigned...* | Lupa `.value` | Tambahkan `.value` atau pakai getter |
| Layar tidak berubah walau nilai sudah diganti | Rx diganti objek baru (`skor = 0.obs`) atau field objek di RxList diubah | Rx selalu `final`; ganti elemen dengan `copyWith` atau panggil `refresh()` |
| *"X" not found. You need to call "Get.put(X())"...* | `Get.find` sebelum didaftarkan, atau controller halaman sudah dihapus | Daftarkan lewat `Get.put`/binding; periksa urutan |
| Request berulang tanpa henti | `muat()` dipanggil di `build`/Obx | Pindahkan ke `onInit` |
| Log/timer tetap jalan setelah halaman ditutup | Lupa membatalkan di `onClose` | `cancel()`/`dispose()` di `onClose` |
| Controller tidak bisa diuji tanpa widget | Controller memanggil `Get.snackbar`/`Get.to` | Kembalikan `bool`/pesan, View yang bereaksi |
| *You are trying to use contextless navigation without a GetMaterialApp* | Memakai `MaterialApp` | Ganti dengan `GetMaterialApp` |

## 12. Studi Kasus: Aplikasi Tugas Kuliah Multi-Halaman

Studi kasus penutup: aplikasi **Tugas Kuliah** (todo) dengan fitur tambah, centang selesai, hapus (geser), filter, badge jumlah tugas aktif, dan halaman statistik. Satu `TodoController` dipakai bersama oleh tiga View, dan semuanya disusun dengan pola MVC.

### Struktur folder

```
lib/
├── main.dart
├── models/todo.dart                  # M
├── controllers/todo_controller.dart  # C
└── views/                            # V
    ├── todo_list_view.dart
    ├── tambah_todo_view.dart
    └── statistik_view.dart
```

Data masih disimpan di memori, jadi belum ada folder `services/`. Saat todo disimpan ke penyimpanan lokal (Modul 10–11) atau Supabase (Modul 12), cukup tambahkan service dan suntikkan ke controller seperti di section 8. View tidak perlu diubah.

### Langkah 1: model

**`lib/models/todo.dart`**

```dart
// lib/models/todo.dart
class Todo {
  final String id;
  final String judul;
  final String matkul;
  final bool selesai;

  const Todo({
    required this.id,
    required this.judul,
    required this.matkul,
    this.selesai = false,
  });

  // Membuat salinan dengan sebagian field diubah (objek tetap immutable)
  Todo copyWith({String? judul, String? matkul, bool? selesai}) {
    return Todo(
      id: id,
      judul: judul ?? this.judul,
      matkul: matkul ?? this.matkul,
      selesai: selesai ?? this.selesai,
    );
  }
}

enum FilterTodo { semua, aktif, selesai }
```

`copyWith` membuat salinan objek dengan sebagian field diganti. Objek `Todo` tidak pernah diubah langsung; yang diganti adalah elemen di `RxList`, sehingga `Obx` pasti tahu ada perubahan (ingat kesalahan nomor 4 di section 11).

### Langkah 2: controller

**`lib/controllers/todo_controller.dart`**

```dart
// lib/controllers/todo_controller.dart
import 'package:get/get.dart';

import '../models/todo.dart';

class TodoController extends GetxController {
  final _todos = <Todo>[
    const Todo(id: '1', judul: 'Laporan praktikum Flutter', matkul: 'Pemrograman Mobile'),
    const Todo(id: '2', judul: 'Normalisasi tabel KRS', matkul: 'Basis Data', selesai: true),
  ].obs;
  final _filter = FilterTodo.semua.obs;

  // ---------- GETTER (data turunan dihitung, tidak disimpan) ----------
  FilterTodo get filter => _filter.value;
  int get jumlahTotal => _todos.length;
  int get jumlahSelesai => _todos.where((t) => t.selesai).length;
  int get jumlahAktif => jumlahTotal - jumlahSelesai;
  double get progres => jumlahTotal == 0 ? 0 : jumlahSelesai / jumlahTotal;

  List<Todo> get todosTampil {
    switch (_filter.value) {
      case FilterTodo.semua:
        return List.unmodifiable(_todos);
      case FilterTodo.aktif:
        return _todos.where((t) => !t.selesai).toList();
      case FilterTodo.selesai:
        return _todos.where((t) => t.selesai).toList();
    }
  }

  Todo? cariById(String id) => _todos.firstWhereOrNull((t) => t.id == id);

  // ---------- AKSI (cukup ubah observable, Obx otomatis rebuild) ----------
  void tambah(String judul, String matkul) {
    final id = DateTime.now().microsecondsSinceEpoch.toString();
    _todos.insert(0, Todo(id: id, judul: judul, matkul: matkul));
  }

  void toggle(String id) {
    final i = _todos.indexWhere((t) => t.id == id);
    if (i == -1) return;
    // ganti elemen dengan salinan baru, bukan mengubah field-nya
    _todos[i] = _todos[i].copyWith(selesai: !_todos[i].selesai);
  }

  void hapus(String id) => _todos.removeWhere((t) => t.id == id);

  void hapusYangSelesai() => _todos.removeWhere((t) => t.selesai);
```

```
  void ubahFilter(FilterTodo f) => _filter.value = f;
}
```

- Hanya dua observable: `_todos` (`RxList`) dan `_filter` (`Rx<FilterTodo>`, dibuat dengan `FilterTodo.semua.obs`).
- Data turunan (`jumlahAktif`, `progres`, `todosTampil`) berupa getter yang dihitung dari `_todos`, tidak disimpan terpisah, sehingga tidak mungkin "tidak sinkron". Obx yang memanggil getter ini tetap reaktif karena getter membaca observable di dalamnya.
- Setiap aksi cukup mengubah observable. Tidak ada `update()` yang bisa terlupa seperti pada `GetBuilder` (section 5).
- Controller hanya mengimpor `package:get/get.dart` dan model. Tidak ada `material.dart`.

### Langkah 3: main.dart

`lib/main.dart`

```dart
// lib/main.dart
import 'package:flutter/material.dart';
import 'package:get/get.dart';

import 'controllers/todo_controller.dart';
import 'views/todo_list_view.dart';

void main() {
  runApp(const TodoApp());
}

class TodoApp extends StatelessWidget {
  const TodoApp({super.key});

  @override
  Widget build(BuildContext context) {
    return GetMaterialApp(
      title: 'Tugas Kuliah',
      debugShowCheckedModeBanner: false,
      theme: ThemeData(colorSchemeSeed: Colors.teal),
      // Satu TodoController untuk seluruh aplikasi
      initialBinding: BindingsBuilder(() {
        Get.put(TodoController(), permanent: true);
      }),
      home: const TodoListView(),
    );
  }
}
```

Karena binding-nya hanya satu baris, dipakai `BindingsBuilder` (section 7) alih-alih membuat class `AppBinding`.

### Langkah 4: halaman daftar

`lib/views/todo_list_view.dart`

```dart
// lib/views/todo_list_view.dart
import 'package:flutter/material.dart';
import 'package:get/get.dart';

import '../controllers/todo_controller.dart';
import '../models/todo.dart';
import 'statistik_view.dart';
import 'tambah_todo_view.dart';

class TodoListView extends GetView<TodoController> {
  const TodoListView({super.key});

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(
        title: const Text('Tugas Kuliah'),
        actions: [
          IconButton(
            tooltip: 'Statistik',
            // Obx kecil: hanya badge yang rebuild saat jumlah aktif berubah
            icon: Obx(
              () => Badge(
                label: Text('${controller.jumlahAktif}'),
                child: const Icon(Icons.insights),
              ),
            ),
            onPressed: () => Get.to(() => const StatistikView()),
          ),
        ],
      ),
      body: const Column(
        children: [
          _FilterBar(),
          Expanded(child: _DaftarTodo()),
        ],
      ),
      floatingActionButton: FloatingActionButton.extended(
        icon: const Icon(Icons.add),
        label: const Text('Tugas Baru'),
        onPressed: () => Get.to(() => const TambahTodoView()),
      ),
    );
  }
}

class _FilterBar extends GetView<TodoController> {
  const _FilterBar();

  @override
  Widget build(BuildContext context) {
    return Padding(
      padding: const EdgeInsets.all(12),
      // Obx ini hanya membaca filter, tidak ikut rebuild saat todo dicentang
```

```
      child: Obx(
        () => SegmentedButton<FilterTodo>(
          segments: const [
            ButtonSegment(value: FilterTodo.semua, label: Text('Semua')),
            ButtonSegment(value: FilterTodo.aktif, label: Text('Aktif')),
            ButtonSegment(value: FilterTodo.selesai, label: Text('Selesai')),
          ],
          selected: {controller.filter},
          onSelectionChanged: (pilihan) => controller.ubahFilter(pilihan.first),
        ),
      ),
    );
  }
}

class _DaftarTodo extends GetView<TodoController> {
  const _DaftarTodo();

  @override
  Widget build(BuildContext context) {
    return Obx(() {
      final todos = controller.todosTampil;

      if (todos.isEmpty) {
        return const Center(child: Text('Tidak ada tugas di filter ini'));
      }

      return ListView.builder(
        itemCount: todos.length,
        itemBuilder: (context, i) {
          final t = todos[i];
          return Dismissible(
            key: ValueKey(t.id),
            direction: DismissDirection.endToStart,
            background: Container(
              color: Colors.red,
              alignment: Alignment.centerRight,
              padding: const EdgeInsets.only(right: 20),
              child: const Icon(Icons.delete, color: Colors.white),
            ),
            onDismissed: (_) {
              controller.hapus(t.id);
              ScaffoldMessenger.of(context).showSnackBar(
                SnackBar(content: Text('"${t.judul}" dihapus')),
              );
            },
            child: CheckboxListTile(
              value: t.selesai,
              onChanged: (_) => controller.toggle(t.id),
              title: Text(
                t.judul,
                style: TextStyle(
                  decoration: t.selesai ? TextDecoration.lineThrough : null,
                ),
              ),
              subtitle: Text(t.matkul),
            ),
          );
        },
```

```
      );
    });
  }
}
```

### Langkah 5: halaman tambah

`lib/views/tambah_todo_view.dart`

```dart
// lib/views/tambah_todo_view.dart
import 'package:flutter/material.dart';
import 'package:get/get.dart';

import '../controllers/todo_controller.dart';

// StatefulWidget: isi form (judul, matkul terpilih) adalah state LOKAL
class TambahTodoView extends StatefulWidget {
  const TambahTodoView({super.key});

  @override
  State<TambahTodoView> createState() => _TambahTodoViewState();
}

class _TambahTodoViewState extends State<TambahTodoView> {
  final _formKey = GlobalKey<FormState>();
  final _judulController = TextEditingController();
  String _matkul = 'Pemrograman Mobile';

  static const daftarMatkul = ['Pemrograman Mobile', 'Basis Data', 'Jaringan Komputer'];

  @override
  void dispose() {
    _judulController.dispose();
    super.dispose();
  }

  void _simpan() {
    if (!_formKey.currentState!.validate()) return;
    // Tidak perlu mengembalikan data lewat Get.back(result: ...): controller yang menyimpan
    Get.find<TodoController>().tambah(_judulController.text.trim(), _matkul);
    Get.back();
  }

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(title: const Text('Tugas Baru')),
      body: Form(
        key: _formKey,
        child: ListView(
          padding: const EdgeInsets.all(16),
          children: [
            TextFormField(
              controller: _judulController,
              decoration: const InputDecoration(
                labelText: 'Judul tugas',
                border: OutlineInputBorder(),
              ),
```

```
              validator: (v) =>
                  (v == null || v.trim().isEmpty) ? 'Judul wajib diisi' : null,
            ),
            const SizedBox(height: 16),
            DropdownButtonFormField<String>(
              initialValue: _matkul,
              decoration: const InputDecoration(
                labelText: 'Mata kuliah',
                border: OutlineInputBorder(),
              ),
              items: [
                for (final m in daftarMatkul) DropdownMenuItem(value: m, child: Text(m)),
              ],
              onChanged: (v) => setState(() => _matkul = v ?? _matkul),
            ),
            const SizedBox(height: 24),
            FilledButton(onPressed: _simpan, child: const Text('Simpan')),
          ],
        ),
      ),
    );
  }
}
```

### Langkah 6: halaman statistik

**lib/views/statistik_view.dart**

```
// lib/views/statistik_view.dart
import 'package:flutter/material.dart';
import 'package:get/get.dart';

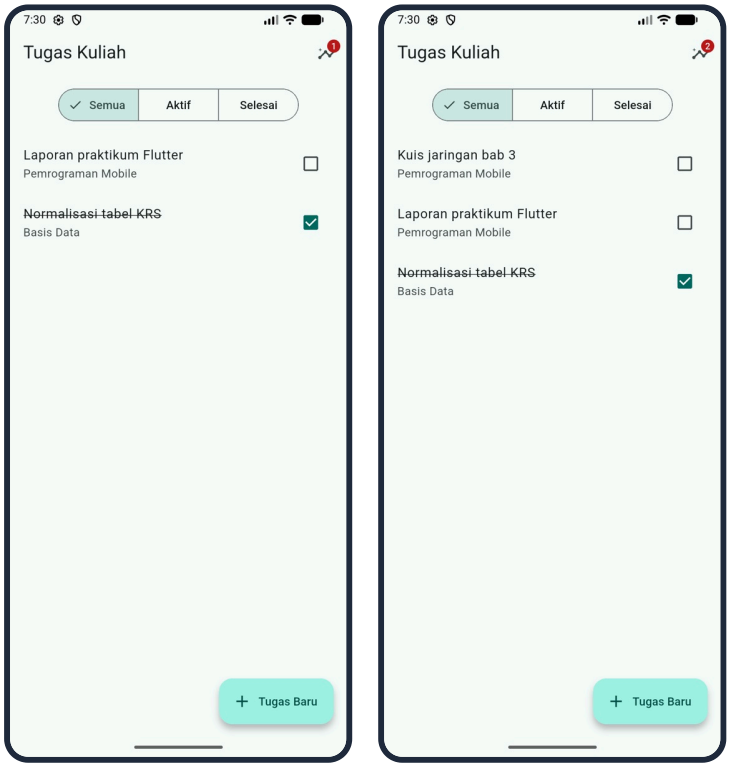
import '../controllers/todo_controller.dart';

class StatistikView extends GetView<TodoController> {
  const StatistikView({super.key});

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(title: const Text('Statistik')),
      body: Padding(
        padding: const EdgeInsets.all(16),
        child: Obx(
          () => Column(
            crossAxisAlignment: CrossAxisAlignment.stretch,
            children: [
              Text(
                '${(controller.progres * 100).round()}% selesai',
                style: Theme.of(context).textTheme.headlineMedium,
              ),
              const SizedBox(height: 8),
              LinearProgressIndicator(value: controller.progres, minHeight: 10),
              const SizedBox(height: 16),
              Card(
                child: Column(
                  children: [
                    ListTile(
```

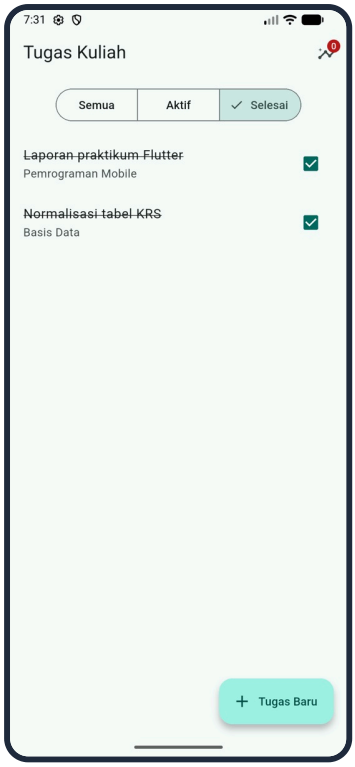
```
                  title: const Text('Total tugas'),
                  trailing: Text('${controller.jumlahTotal}'),
                ),
                ListTile(
                  title: const Text('Aktif'),
                  trailing: Text('${controller.jumlahAktif}'),
                ),
                ListTile(
                  title: const Text('Selesai'),
                  trailing: Text('${controller.jumlahSelesai}'),
                ),
              ],
            ),
          ),
          const SizedBox(height: 16),
          OutlinedButton.icon(
            icon: const Icon(Icons.cleaning_services),
            label: const Text('Hapus semua yang selesai'),
            onPressed: controller.jumlahSelesai == 0
                ? null
                : controller.hapusYangSelesai,
          ),
        ],
      ),
    ),
  ),
);
  }
}
```

## Hasil

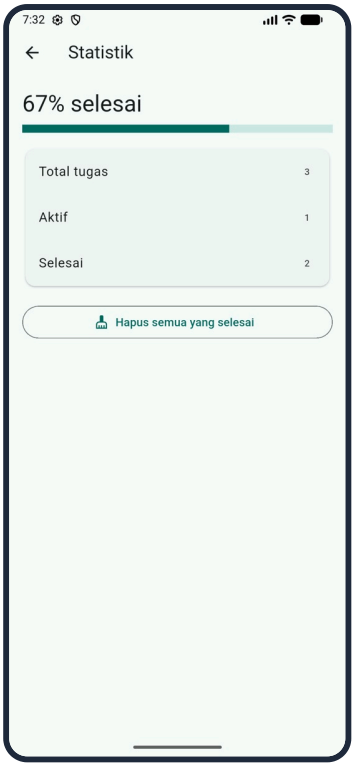



Daftar tugas awal

Tugas baru muncul, badge aktif bertambah




Filter Selesai setelah satu tugas dicentang




Halaman statistik membaca controller yang sama

**Ringkasan teknik yang dipakai:**

| Bagian | Teknik | Alasan |
|---|---|---|
| Badge statistik di AppBar | `Obx` kecil hanya di sekitar `Badge` | Hanya ikon yang rebuild saat jumlah aktif berubah |
| Filter | `Obx` yang hanya membaca `controller.filter` | Tombol filter tidak rebuild saat todo dicentang |
| Daftar todo | `Obx` yang membaca `todosTampil` | Butuh seluruh daftar terkini sesuai filter |
| Mengambil controller | `GetView<TodoController>` / `Get.find` | Tiga View memakai objek yang sama tanpa dioper lewat constructor |
| Tambah tugas | `controller.tambah(...)` lalu `Get.back()` | Tidak perlu `Get.back(result: ...)`; controller yang menyimpan |
| Centang | Ganti elemen dengan `copyWith` | RxList mendeteksi perubahan elemen |
| Hapus | `Dismissible` + `ValueKey(t.id)` | Key unik agar Flutter tidak salah menghapus item |
| Form tambah | `StatefulWidget` + `TextEditingController` | Isi form adalah state lokal, tidak perlu masuk controller |

**ℹ️ Bandingkan dengan Modul 07**

Di studi kasus Modul 07, data KRS harus dioper lewat constructor dan callback, dan halaman tambah mengembalikan hasil lewat `Navigator.pop`. Di sini, ketiga View cukup mengambil `TodoController` dengan `Get.find` / `GetView`, dan setiap perubahan langsung terlihat di semua halaman berkat `Obx`. Pola yang sama dipakai di proyek CRUD Modul 13–14, dengan tambahan service, status loading/error dari section 8, dan auth gate dari section 10.

💡 **Tantangan tambahan**

- Tambahkan halaman edit yang memakai `cariById` dan method baru `ubah(id, judul)` di controller (ganti elemen dengan `copyWith`).
- Tambahkan tombol "Urungkan" pada SnackBar hapus. Petunjuk: buat method `hapus` yang mengembalikan `Todo` yang dihapus beserta indeksnya, lalu `sisipkan(index, todo)`.
- Tambahkan kotak pencarian judul tugas dengan worker `debounce` (section 9).
- Simpan todo ke penyimpanan lokal (Modul 10/11): buat `TodoService`, suntikkan ke controller, dan muat datanya di `onInit`.
- Pisahkan binding ke `lib/bindings/app_binding.dart` seperti section 10.

# Tugas Praktikum: Praktikum 8: Sistem Pengambilan SKS & Keranjang Rencana Studi (GetX MVC)

**Sasaran Praktikum**

Menerapkan GetX dengan pola MVC untuk sinkronisasi state keranjang KRS antar beberapa layar secara real-time, lengkap dengan pemuatan data async dan struktur folder yang rapi.

## Langkah Pengerjaan

1. Tambahkan package `get` ke dalam proyek menggunakan perintah: `flutter pub add get`, lalu ganti `MaterialApp` menjadi `GetMaterialApp`.
2. Susun folder `lib/models`, `lib/services`, `lib/controllers`, `lib/bindings`, `lib/views`, dan `lib/widgets`.
3. Buat model `MataKuliah` dan `MatkulService` yang mengembalikan daftar mata kuliah setelah `Future.delayed` 2 detik.
4. Buat `KatalogController` dengan pola `isLoading` / `error` / `data` berbasis `.obs`. Panggil pemuatan data di `onInit()`. Tampilkan loading, error + tombol Coba Lagi, dan `RefreshIndicator`.
5. Buat `KrsController` yang mencatat mata kuliah yang dipilih (`RxList`), total SKS (getter), dan sisa kuota SKS. Getter list harus mengembalikan `List.unmodifiable`.
6. Daftarkan kedua controller dalam satu `AppBinding extends Bindings` dan pasang di `GetMaterialApp(initialBinding: AppBinding())`.
7. Buat View 1 (Katalog Matkul): menampilkan daftar mata kuliah dengan tombol '+ Ambil Matkul' yang memanggil `Get.find<KrsController>().tambahMatkul(...)`.
8. Cegah penambahan jika kuota melebihi 24 SKS atau mata kuliah sudah diambil. Controller mengembalikan hasil, View yang menampilkan SnackBar peringatan.
9. Buat View 2 (Kartu Rencana Studi): menampilkan daftar mata kuliah yang sudah diambil, tombol hapus dengan dialog konfirmasi, dan total kalkulasi SKS.
10. Pasang badge angka di ikon tab bar yang otomatis bertambah saat mata kuliah baru diambil. Gunakan `Obx` kecil yang hanya membungkus badge agar hanya badge yang rebuild.
11. Pastikan controller tidak mengimpor `material.dart` dan tidak memegang `BuildContext`.
12. Bonus: tambahkan `AuthController` sederhana dan `AuthGate` berbasis `Obx` yang menampilkan layar login sebelum katalog.

### Output yang Diharapkan

Dua atau lebih halaman yang saling terhubung secara reaktif tanpa perlu restart halaman; saat mata kuliah diambil di halaman 1, angka di halaman 2 dan badge tab langsung bertambah. Katalog menampilkan loading, error, dan data dengan benar.

### Rubrik Penilaian

Desain GetxController dan state reaktif .obs (25%), Penggunaan Obx dan dependency injection Get.put/Get.find/Bindings yang tepat (25%), Pola async isLoading/error/data di onInit (20%), Pencegahan over-quota 24 SKS dan feedback UI di View (15%), Struktur folder MVC models/services/controllers/views (15%). Bonus AuthGate +10%.

## Kuis Pemahaman

**1.** Dalam pola MVC dengan GetX, di lapisan mana logika 'tolak penambahan jika total SKS melebihi 24' sebaiknya ditulis?

**A.** Controller (KrsController)

**B.** View (KatalogView)

**C.** Model (MataKuliah)

**D.** main.dart

**2.** Apa yang terjadi ketika nilai `count.value` pada `final count = 0.obs;` diubah?

**A.** Semua Obx yang membaca count otomatis di-build ulang

**B.** Seluruh aplikasi di-restart

**C.** Tidak terjadi apa-apa sampai setState() dipanggil

**D.** Controller dihapus dari memori

**3.** Sebuah widget hanya membutuhkan totalSks dan tidak boleh ikut rebuild ketika field lain berubah. Cara paling tepat adalah ...

**A.** Membungkus Text totalSks dengan Obx kecil yang hanya membaca controller.totalSks

**B.** Membungkus seluruh Scaffold dengan satu Obx

**C.** Memanggil Get.put(KrsController()) di dalam build

**D.** Mengganti Obx dengan setState

**4.** Muncul error *"[Get] the improper use of a GetX has been detected"*. Penyebab paling mungkin adalah ...

**A.** Di dalam Obx tidak ada satu pun variabel observable (.obs) yang dibaca

**B.** Lupa memanggil flutter pub get

**C.** Controller belum memiliki method onClose

**D.** Memakai GetMaterialApp bukan MaterialApp

**5.** `Get.find<KrsController>()` melempar error "KrsController not found". Perbaikannya adalah ...

**A.** Mendaftarkan controller lebih dulu dengan Get.put, Get.lazyPut, atau Bindings

**B.** Mengganti Get.find dengan Obx

**C.** Menambahkan .obs pada nama class

**D.** Memanggil notifyListeners()

**6.** Di mana tempat paling tepat untuk memanggil pemuatan data awal (misalnya muatKatalog()) pada GetxController?

**A.** Di method onInit() milik controller

**B.** Di dalam build() View

**C.** Di constructor View

**D.** Di onClose()

**7.** Pada pola state async di controller, mengapa `_isLoading.value = false` sebaiknya diletakkan di blok finally?

**A.** Agar loading selalu dimatikan, baik request berhasil maupun gagal

**B.** Agar request berjalan lebih cepat

**C.** Karena Dart melarang assignment di blok try

**D.** Agar error tidak pernah terjadi

**8.** Halaman detail dibuka dengan `Get.to(() => const DetailView())` dan perlu memakai OBJEK controller yang sama dengan halaman daftar. Cara yang benar adalah ...

**A.** Memanggil Get.find<TodoController>() di DetailView

**B.** Membuat TodoController() baru di DetailView

**C.** Mengirim controller lewat Get.back()

**D.** Memanggil Get.put(TodoController()) lagi dengan tag acak

**9.** Dalam struktur folder MVC kursus ini, lapisan mana yang seharusnya memanggil HTTP atau database?

**A.** services

**B.** views

**C.** models

**D.** controllers

## Kunci Jawaban & Pembahasan

**1. A** — Aturan bisnis tinggal di controller supaya berlaku sama untuk semua View yang memanggilnya dan bisa diuji dengan unit test. View hanya menampilkan hasilnya, misalnya lewat SnackBar.

**2. A** — Variabel .obs adalah observable. Setiap Obx yang membaca nilainya saat build terdaftar sebagai pendengar dan otomatis di-build ulang ketika nilainya berubah, tanpa notifyListeners atau setState.

**3. A** — Obx hanya rebuild ketika observable yang dibaca di dalam builder-nya berubah. Obx kecil di sekitar widget yang butuh data membatasi rebuild seminimal mungkin.

**4. A** — Obx harus membaca minimal satu observable di dalam builder-nya. Jika yang dibaca hanya variabel biasa (bukan .obs), GetX menampilkan error ini.

**5. A** — Get.find hanya mengambil instance yang sudah didaftarkan. Controller harus disediakan dulu, misalnya lewat Get.put di main, initialBinding, atau binding pada GetPage.

**6. A** — onInit dipanggil sekali saat controller dibuat. Memanggil pemuatan di build akan mengulang request setiap rebuild. onClose dipakai untuk membersihkan sumber daya, misalnya membatalkan StreamSubscription.

**7. A** — Blok finally selalu dijalankan setelah try/catch. Tanpa finally, jika terjadi exception, isLoading bisa tetap true dan View terjebak di tampilan loading.

**8. A** — Controller yang sudah didaftarkan tersimpan di dependency manager GetX, sehingga halaman mana pun bisa mengambil instance yang sama dengan Get.find tanpa mengopernya lewat constructor.

**9. A** — Service adalah satu-satunya lapisan yang tahu cara mengakses data. Controller memanggil service, View memanggil controller. Dengan begitu sumber data mudah diganti dan diuji.